

# ClusterSetup

Cisco Ramon

*[2026-03-14 Sat 00:35]*

## Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                             | <b>12</b> |
| <b>2</b> | <b>Preface</b>                              | <b>12</b> |
| <b>3</b> | <b>About the Author</b>                     | <b>14</b> |
| <b>4</b> | <b>Introduction</b>                         | <b>15</b> |
| <b>5</b> | <b>Cluster Architecture</b>                 | <b>17</b> |
| <b>6</b> | <b>Node Details</b>                         | <b>17</b> |
| 6.1      | Node Role . . . . .                         | 17        |
| 6.2      | Networking Setup . . . . .                  | 18        |
| 6.3      | Static IP Addressing . . . . .              | 19        |
| <b>7</b> | <b>Resource Allocation</b>                  | <b>19</b> |
| 7.1      | Master Node . . . . .                       | 19        |
| 7.2      | Login Node . . . . .                        | 19        |
| 7.3      | Compute Nodes . . . . .                     | 19        |
| <b>8</b> | <b>Master Node Setup</b>                    | <b>20</b> |
| 8.1      | Install VirtualBox on Manjaro . . . . .     | 20        |
| 8.2      | Download Rocky Linux . . . . .              | 21        |
| 8.3      | Create Master VM . . . . .                  | 21        |
| 8.4      | Network Configuration . . . . .             | 22        |
| 8.5      | Rocky Linux Installation . . . . .          | 22        |
| 8.6      | Initial System Verification . . . . .       | 23        |
| 8.7      | Configure Static Internal Network . . . . . | 23        |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| 8.8       | Configure Internal Interface . . . . .               | 24        |
| 8.9       | Restart Networking . . . . .                         | 24        |
| 8.10      | Verify Networking . . . . .                          | 24        |
| 8.11      | SSH Access from Host . . . . .                       | 24        |
| 8.12      | System Update and Packages . . . . .                 | 25        |
| 8.13      | Timezone Configuration . . . . .                     | 25        |
| 8.14      | Configure /etc/hosts . . . . .                       | 25        |
| 8.15      | Disable Firewall (Temporary) . . . . .               | 26        |
| 8.16      | SELinux Configuration . . . . .                      | 26        |
| 8.17      | Verify Hostname . . . . .                            | 26        |
| <b>9</b>  | <b>Cloning Nodes</b>                                 | <b>27</b> |
| 9.1       | Create Snapshot . . . . .                            | 27        |
| 9.2       | Clone Nodes . . . . .                                | 27        |
| 9.3       | Clone Names . . . . .                                | 27        |
| 9.4       | Configure Port Forwarding . . . . .                  | 27        |
| 9.5       | Machine ID Regeneration . . . . .                    | 28        |
| 9.6       | Remove Existing Machine ID . . . . .                 | 28        |
| 9.7       | Generate New Machine ID . . . . .                    | 28        |
| 9.8       | Verify Machine ID . . . . .                          | 28        |
| 9.9       | Configure Hostname . . . . .                         | 28        |
| 9.10      | Configure Static Internal IPs . . . . .              | 29        |
| 9.11      | Restart Networking . . . . .                         | 30        |
| 9.12      | Verify Networking . . . . .                          | 30        |
| 9.13      | Verify Routing . . . . .                             | 30        |
| 9.14      | Verify Connectivity . . . . .                        | 30        |
| 9.15      | SSH Verification . . . . .                           | 31        |
| <b>10</b> | <b>Shared Infrastructure Services</b>                | <b>31</b> |
| 10.1      | Cluster Role Architecture . . . . .                  | 32        |
| 10.2      | Verify User Consistency . . . . .                    | 33        |
| 10.3      | Passwordless SSH Configuration . . . . .             | 33        |
| 10.4      | Generate SSH Key on Login Node . . . . .             | 33        |
| 10.5      | Copy SSH Keys to Cluster Nodes . . . . .             | 34        |
| 10.6      | Verify Passwordless SSH . . . . .                    | 34        |
| 10.7      | HPC Concept — Why Passwordless SSH Matters . . . . . | 35        |
| 10.8      | NFS Shared Storage Setup . . . . .                   | 35        |
| 10.9      | Install NFS Packages . . . . .                       | 35        |
| 10.10     | Configure NFS Server on Master . . . . .             | 36        |
| 10.11     | Enable NFS Services . . . . .                        | 36        |

|           |                                                                                 |           |
|-----------|---------------------------------------------------------------------------------|-----------|
| 10.12     | Configure NFS Exports . . . . .                                                 | 36        |
| 10.13     | Export Option Explanation . . . . .                                             | 36        |
| 10.14     | Apply Exports . . . . .                                                         | 36        |
| 10.15     | Verify Exports . . . . .                                                        | 36        |
| 10.16     | Configure NFS Clients . . . . .                                                 | 37        |
| 10.17     | Backup Existing Home Directory . . . . .                                        | 37        |
| 10.18     | Create New Mount Point . . . . .                                                | 37        |
| 10.19     | Mount Shared NFS Home . . . . .                                                 | 37        |
| 10.20     | Verify Mount . . . . .                                                          | 37        |
| 10.21     | Verify Shared Filesystem . . . . .                                              | 38        |
| 10.22     | Configure Persistent NFS Mounts . . . . .                                       | 38        |
| 10.23     | Configure /etc/fstab . . . . .                                                  | 38        |
| 10.24     | Verify fstab Before Reboot . . . . .                                            | 38        |
| 10.25     | Reboot Validation Procedure . . . . .                                           | 39        |
| 10.26     | Reboot Order . . . . .                                                          | 39        |
| 10.27     | Verify Persistent Mounts After Reboot . . . . .                                 | 39        |
| 10.28     | Verify Active NFS Mount . . . . .                                               | 40        |
| 10.29     | Verify Shared Files After Reboot . . . . .                                      | 40        |
| <b>11</b> | <b>Munge Authentication + Slurm Scheduler</b>                                   | <b>40</b> |
| 11.1      | HPC Scheduler Architecture . . . . .                                            | 41        |
| 11.2      | IMPORTANT Rocky Linux 10 Issue . . . . .                                        | 42        |
| 11.3      | Install EPEL Repository . . . . .                                               | 42        |
| 11.4      | Enable CRB Repository . . . . .                                                 | 42        |
| 11.5      | Install Munge + Slurm Packages . . . . .                                        | 43        |
| 11.6      | Verify Installed Packages . . . . .                                             | 43        |
| 11.7      | Verify System Users . . . . .                                                   | 43        |
| 11.8      | IMPORTANT Issue Encountered — Missing slurm User .                              | 44        |
| 11.9      | Create Slurm Group . . . . .                                                    | 44        |
| 11.10     | Create Slurm User . . . . .                                                     | 44        |
| 11.11     | Verify Slurm User . . . . .                                                     | 44        |
| 11.12     | IMPORTANT HPC Security Concept . . . . .                                        | 44        |
| 11.13     | Configure Munge Authentication . . . . .                                        | 45        |
| 11.14     | Generate Munge Key on Master . . . . .                                          | 45        |
| 11.15     | Verify Munge Key . . . . .                                                      | 45        |
| 11.16     | IMPORTANT Mistake Encountered — Munge Key Gen-<br>erated on All Nodes . . . . . | 45        |
| 11.17     | Correct Munge Key Distribution . . . . .                                        | 46        |
| 11.18     | IMPORTANT SSH Root Login Issue . . . . .                                        | 46        |
| 11.19     | Copy Munge Key to Login Node . . . . .                                          | 46        |

|       |                                                                              |    |
|-------|------------------------------------------------------------------------------|----|
| 11.20 | Copy Munge Key to Compute01 . . . . .                                        | 46 |
| 11.21 | Copy Munge Key to Compute02 . . . . .                                        | 46 |
| 11.22 | Copy Munge Key to Compute03 . . . . .                                        | 47 |
| 11.23 | Remove Temporary Key Copy . . . . .                                          | 47 |
| 11.24 | Install Munge Key on Client Nodes . . . . .                                  | 47 |
| 11.25 | Stop Munge Service . . . . .                                                 | 47 |
| 11.26 | Install Key . . . . .                                                        | 47 |
| 11.27 | Fix Ownership . . . . .                                                      | 47 |
| 11.28 | Fix Permissions . . . . .                                                    | 47 |
| 11.29 | Start Munge . . . . .                                                        | 47 |
| 11.30 | IMPORTANT Issue Encountered — Munge Not Running<br>on Master . . . . .       | 48 |
| 11.31 | Verify Munge Service . . . . .                                               | 48 |
| 11.32 | Verify Munge Authentication . . . . .                                        | 48 |
| 11.33 | Test Compute02 . . . . .                                                     | 48 |
| 11.34 | Test Compute03 . . . . .                                                     | 48 |
| 11.35 | Test Login Node . . . . .                                                    | 49 |
| 11.36 | HPC Concepts Learned . . . . .                                               | 49 |
| 11.37 | Configure Slurm Scheduler . . . . .                                          | 49 |
| 11.38 | IMPORTANT Issue Encountered — Default localhost Con-<br>figuration . . . . . | 49 |
| 11.39 | Create Slurm Configuration . . . . .                                         | 50 |
| 11.40 | Configuration Explanation . . . . .                                          | 51 |
| 11.41 | Create Slurm Runtime Directories . . . . .                                   | 51 |
| 11.42 | Set Ownership . . . . .                                                      | 51 |
| 11.43 | Create slurmd Directories on Compute Nodes . . . . .                         | 51 |
| 11.44 | IMPORTANT sudo Over SSH Issue . . . . .                                      | 52 |
| 11.45 | Fix Ownership on Compute Nodes . . . . .                                     | 52 |
| 11.46 | Verify Runtime Directories . . . . .                                         | 52 |
| 11.47 | Distribute slurm.conf . . . . .                                              | 53 |
| 11.48 | Install Configuration on Nodes . . . . .                                     | 53 |
| 11.49 | Start Slurm Controller . . . . .                                             | 53 |
| 11.50 | Verify Controller . . . . .                                                  | 53 |
| 11.51 | Start slurmd on Compute Nodes . . . . .                                      | 54 |
| 11.52 | Verify slurmd . . . . .                                                      | 54 |
| 11.53 | Reconfigure Slurm . . . . .                                                  | 54 |
| 11.54 | IMPORTANT Issue Encountered — Nodes in DRAIN State                           | 54 |
| 11.55 | Stop slurmd . . . . .                                                        | 55 |
| 11.56 | Remove Old State . . . . .                                                   | 55 |
| 11.57 | Restart slurmd . . . . .                                                     | 55 |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| 11.58     | Resume Nodes . . . . .                               | 55        |
| 11.59     | Verify Cluster Status . . . . .                      | 55        |
| 11.60     | Verify Node Details . . . . .                        | 55        |
| 11.61     | Test Slurm Execution . . . . .                       | 56        |
| 11.62     | Multi-node Execution . . . . .                       | 56        |
| 11.63     | Multi-task Multi-node Execution . . . . .            | 56        |
| 11.64     | Multi-core Single-node Execution . . . . .           | 56        |
| 11.65     | IMPORTANT Slurm Scheduling Concept . . . . .         | 57        |
| <b>12</b> | <b>Shared Software Filesystem (/apps)</b>            | <b>57</b> |
| 12.1      | Why Separate /apps Filesystem? . . . . .             | 58        |
| 12.2      | IMPORTANT Storage Architecture Concept . . . . .     | 58        |
| 12.3      | Add New Virtual Disk to Master . . . . .             | 59        |
| 12.4      | Verify New Disk Detection . . . . .                  | 60        |
| 12.5      | Create GPT Partition Table . . . . .                 | 60        |
| 12.6      | Verify Partition . . . . .                           | 61        |
| 12.7      | Create XFS Filesystem . . . . .                      | 61        |
| 12.8      | Create Mount Point . . . . .                         | 61        |
| 12.9      | Get Filesystem UUID . . . . .                        | 61        |
| 12.10     | Configure Persistent Mount . . . . .                 | 61        |
| 12.11     | Mount Filesystem . . . . .                           | 62        |
| 12.12     | Verify Mount . . . . .                               | 62        |
| 12.13     | Configure Ownership . . . . .                        | 62        |
| 12.14     | Create Shared HPC Software Hierarchy . . . . .       | 62        |
| 12.15     | HPC Software Hierarchy Concept . . . . .             | 63        |
| 12.16     | Export /apps via NFS . . . . .                       | 63        |
| 12.17     | Apply NFS Exports . . . . .                          | 63        |
| 12.18     | Verify Exports . . . . .                             | 63        |
| 12.19     | Create Mount Points on Client Nodes . . . . .        | 63        |
| 12.20     | Configure Persistent NFS Mounts . . . . .            | 64        |
| 12.21     | Mount Shared /apps Filesystem . . . . .              | 64        |
| 12.22     | Verify Shared Mount . . . . .                        | 64        |
| 12.23     | Verify Shared Filesystem Functionality . . . . .     | 64        |
| 12.24     | HPC Architecture After /apps Setup . . . . .         | 65        |
| <b>13</b> | <b>Spack Software Stack + MPI Runtime Validation</b> | <b>65</b> |
| 13.1      | IMPORTANT HPC Software Stack Concept . . . . .       | 66        |
| 13.2      | IMPORTANT Shared Filesystem Concept . . . . .        | 67        |
| 13.3      | Install Development Tools . . . . .                  | 67        |
| 13.4      | Move into Shared Application Filesystem . . . . .    | 68        |

|       |                                                       |    |
|-------|-------------------------------------------------------|----|
| 13.5  | Clone Spack into Shared Filesystem . . . . .          | 68 |
| 13.6  | Enter Spack Directory . . . . .                       | 69 |
| 13.7  | Initialize Spack Environment . . . . .                | 69 |
| 13.8  | IMPORTANT Shell Environment Concept . . . . .         | 70 |
| 13.9  | Make Spack Permanent . . . . .                        | 70 |
| 13.10 | Clone Additional Spack Repository . . . . .           | 71 |
| 13.11 | Configure Spack Repository . . . . .                  | 71 |
| 13.12 | Verify Spack Functionality . . . . .                  | 71 |
| 13.13 | IMPORTANT HPC Software Stack Concept . . . . .        | 72 |
| 13.14 | IMPORTANT Spack Build Concept . . . . .               | 72 |
| 13.15 | Detect Available Compilers . . . . .                  | 73 |
| 13.16 | Install GCC Using Spack . . . . .                     | 73 |
| 13.17 | IMPORTANT Multiple Compiler Version Concept . . . . . | 73 |
| 13.18 | Verify Installed Software . . . . .                   | 74 |
| 13.19 | Load Installed GCC Compiler . . . . .                 | 74 |
| 13.20 | IMPORTANT Runtime Environment Concept . . . . .       | 75 |
| 13.21 | Register Newly Installed Compiler . . . . .           | 75 |
| 13.22 | Install OpenMPI Using Spack . . . . .                 | 75 |
| 13.23 | IMPORTANT MPI Compiler Dependency Concept . . . . .   | 76 |
| 13.24 | Verify Installed Software Tree . . . . .              | 76 |
| 13.25 | IMPORTANT Spack Spec Concept . . . . .                | 77 |
| 13.26 | IMPORTANT Hash-Based Software Loading . . . . .       | 77 |
| 13.27 | Load OpenMPI Environment . . . . .                    | 78 |
| 13.28 | IMPORTANT MPI Wrapper Concept . . . . .               | 78 |
| 13.29 | Verify Loaded Software . . . . .                      | 78 |
| 13.30 | Create MPI Performance Validation Program . . . . .   | 79 |
| 13.31 | IMPORTANT MPI Reduction Concept . . . . .             | 82 |
| 13.32 | IMPORTANT Parallel Execution Concept . . . . .        | 83 |
| 13.33 | Load Software Before Compilation . . . . .            | 83 |
| 13.34 | Compile MPI Program . . . . .                         | 84 |
| 13.35 | IMPORTANT MPI Compilation Concept . . . . .           | 84 |
| 13.36 | Interactive MPI Runtime Validation . . . . .          | 85 |
| 13.37 | IMPORTANT Shared Filesystem Advantage . . . . .       | 85 |
| 13.38 | Create Slurm Batch Script . . . . .                   | 86 |
| 13.39 | IMPORTANT Slurm Script Explanation . . . . .          | 86 |
| 13.40 | IMPORTANT Runtime Environment Section . . . . .       | 87 |
| 13.41 | IMPORTANT Slurm Runtime Variable . . . . .            | 87 |
| 13.42 | Submit MPI Job . . . . .                              | 88 |
| 13.43 | Monitor Jobs . . . . .                                | 88 |
| 13.44 | IMPORTANT Job States . . . . .                        | 88 |

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| 13.45     | View Detailed Job Information . . . . .                           | 89        |
| 13.46     | Check Cluster Status . . . . .                                    | 89        |
| 13.47     | IMPORTANT Node States . . . . .                                   | 89        |
| 13.48     | Check Output Files . . . . .                                      | 90        |
| 13.49     | IMPORTANT HPC Performance Metrics . . . . .                       | 90        |
| 13.50     | Cancel Jobs . . . . .                                             | 91        |
| 13.51     | Interactive Slurm Execution . . . . .                             | 91        |
| 13.52     | IMPORTANT HPC Workflow . . . . .                                  | 92        |
| 13.53     | IMPORTANT Production HPC Concept . . . . .                        | 92        |
| <b>14</b> | <b>Centralized Identity Management using OpenLDAP +<br/>nslcd</b> | <b>93</b> |
| 14.1      | Important Architecture . . . . .                                  | 94        |
| 14.2      | Important Linux Authentication Concepts . . . . .                 | 94        |
| 14.3      | Install OpenLDAP Server on Master Node . . . . .                  | 94        |
| 14.4      | Enable LDAP Server . . . . .                                      | 95        |
| 14.5      | Generate LDAP Admin Password Hash . . . . .                       | 95        |
| 14.6      | Configure LDAP Database . . . . .                                 | 96        |
| 14.7      | Important LDAP Concepts . . . . .                                 | 97        |
| 14.8      | Apply LDAP Database Configuration . . . . .                       | 97        |
| 14.9      | Import LDAP Schemas . . . . .                                     | 98        |
| 14.10     | Create LDAP Base Tree . . . . .                                   | 98        |
| 14.11     | Verify LDAP Tree . . . . .                                        | 99        |
| 14.12     | Create LDAP Group . . . . .                                       | 100       |
| 14.13     | Create LDAP User Password Hash . . . . .                          | 100       |
| 14.14     | Create LDAP User . . . . .                                        | 101       |
| 14.15     | Important LDAP User Attributes . . . . .                          | 102       |
| 14.16     | Verify LDAP User . . . . .                                        | 102       |
| 14.17     | Configure LDAP Client on Login Node . . . . .                     | 102       |
| 14.18     | Configure nslcd . . . . .                                         | 103       |
| 14.19     | Enable nslcd . . . . .                                            | 103       |
| 14.20     | Verify LDAP Connectivity . . . . .                                | 104       |
| 14.21     | Configure NSS . . . . .                                           | 104       |
| 14.22     | Restart nslcd . . . . .                                           | 105       |
| 14.23     | Verify LDAP User Resolution . . . . .                             | 105       |
| 14.24     | NOTE (Do this for master too) . . . . .                           | 105       |
| 14.25     | Install Automatic Home Directory Support . . . . .                | 105       |
| 14.26     | Configure PAM Authentication . . . . .                            | 106       |
| 14.27     | Configure Second PAM File . . . . .                               | 107       |
| 14.28     | Restart Services . . . . .                                        | 107       |

|           |                                                  |            |
|-----------|--------------------------------------------------|------------|
| 14.29     | Verify LDAP Authentication . . . . .             | 107        |
| 14.30     | Verify User Home . . . . .                       | 108        |
| 14.31     | Troubleshooting . . . . .                        | 108        |
| 14.32     | Current Cluster State After LDAP Setup . . . . . | 113        |
| 14.33     | Important Debugging Commands . . . . .           | 114        |
| 14.34     | Important Architecture Understanding . . . . .   | 114        |
| <b>15</b> | <b>Automated LDAP User Creation Script</b>       | <b>115</b> |
| 15.1      | Script Location . . . . .                        | 115        |
| 15.2      | User Creation Script . . . . .                   | 116        |
| 15.3      | Make Script Executable . . . . .                 | 118        |
| 15.4      | Understanding Script Arguments . . . . .         | 119        |
| 15.5      | Default Password Format . . . . .                | 120        |
| 15.6      | What the Script Automatically Does . . . . .     | 120        |
| 15.7      | Example User Creation . . . . .                  | 120        |
| 15.8      | Verify User Creation . . . . .                   | 121        |
| 15.9      | Verify Login . . . . .                           | 121        |
| 15.10     | Important UID/GID Notes . . . . .                | 122        |
| 15.11     | Important HPC Identity Concept . . . . .         | 122        |
| <b>16</b> | <b>Automated LDAP User Deletion Script</b>       | <b>123</b> |
| 16.1      | Script Location . . . . .                        | 123        |
| 16.2      | User Deletion Script . . . . .                   | 124        |
| 16.3      | Make Script Executable . . . . .                 | 126        |
| 16.4      | Understanding Script Arguments . . . . .         | 126        |
| 16.5      | What the Script Automatically Does . . . . .     | 127        |
| 16.6      | Example User Deletion . . . . .                  | 127        |
| 16.7      | Important Group Deletion Notes . . . . .         | 128        |
| 16.8      | Important Home Directory Notes . . . . .         | 128        |
| 16.9      | Verify User Removal . . . . .                    | 129        |
| 16.10     | Important HPC User Management Concept . . . . .  | 129        |
| <b>17</b> | <b>Automated LDAP Password Change Script</b>     | <b>130</b> |
| 17.1      | Script Location . . . . .                        | 130        |
| 17.2      | Password Change Script . . . . .                 | 131        |
| 17.3      | Make Script Executable . . . . .                 | 133        |
| 17.4      | Understanding Script Arguments . . . . .         | 133        |
| 17.5      | What the Script Automatically Does . . . . .     | 133        |
| 17.6      | Example Password Change . . . . .                | 133        |
| 17.7      | Verify Password Change . . . . .                 | 134        |



|           |                                                               |            |
|-----------|---------------------------------------------------------------|------------|
| 17.8      | Important Password Management Notes . . . . .                 | 135        |
| <b>18</b> | <b>Advanced Slurm Configuration Preparation</b>               | <b>135</b> |
| 18.1      | IMPORTANT APPROACH . . . . .                                  | 136        |
| 18.2      | Advanced Slurm Roadmap . . . . .                              | 136        |
| 18.3      | Resource-Aware Scheduling . . . . .                           | 137        |
| 18.4      | Understanding Current Cluster Resource State . . . . .        | 137        |
| 18.5      | Understanding Actual Node Memory . . . . .                    | 138        |
| 18.6      | Configuring RealMemory . . . . .                              | 139        |
| 18.7      | RealMemory Concept . . . . .                                  | 139        |
| 18.8      | Resource-Aware Scheduling Concept . . . . .                   | 140        |
| 18.9      | Scheduler Configuration Enhancements . . . . .                | 140        |
| 18.10     | Configuration Explanation . . . . .                           | 140        |
| 18.11     | Distributing Updated Slurm Configuration . . . . .            | 142        |
| 18.12     | Restarting Slurm Services . . . . .                           | 142        |
| 18.13     | Verifying Cluster Health . . . . .                            | 143        |
| 18.14     | Difference Between Scheduling and Enforcement . . . . .       | 143        |
| 18.15     | Why task/cgroup Was Postponed . . . . .                       | 144        |
| 18.16     | Process Tracking Note . . . . .                               | 145        |
| 18.17     | Verifying Stable Job Execution . . . . .                      | 146        |
| 18.18     | Milestone Achieved . . . . .                                  | 146        |
| <b>19</b> | <b>Advanced Slurm Configuration</b>                           | <b>147</b> |
| 19.1      | IMPORTANT APPROACH . . . . .                                  | 147        |
| 19.2      | Current Cluster State Before Advanced Configuration . . . . . | 147        |
| 19.3      | Resource-Aware Scheduling . . . . .                           | 148        |
| 19.4      | CPU Affinity and Task Binding . . . . .                       | 152        |
| 19.5      | Understanding CPU Topology . . . . .                          | 153        |
| 19.6      | CPU Affinity Validation Program . . . . .                     | 154        |
| 19.7      | Distributed CPU Placement Test . . . . .                      | 154        |
| 19.8      | Multiple Task Placement Test . . . . .                        | 155        |
| 19.9      | Explicit CPU Binding . . . . .                                | 156        |
| 19.10     | Slurm Accounting Database Setup . . . . .                     | 157        |
| 19.11     | Install MariaDB . . . . .                                     | 158        |
| 19.12     | Secure MariaDB . . . . .                                      | 158        |
| 19.13     | Create Slurm Accounting Database . . . . .                    | 158        |
| 19.14     | Install Slurm Accounting Daemon . . . . .                     | 159        |
| 19.15     | Configure slurmdbd . . . . .                                  | 160        |
| 19.16     | Connect Slurm With Accounting System . . . . .                | 160        |
| 19.17     | Restart Slurm Services . . . . .                              | 161        |

|           |                                                                            |            |
|-----------|----------------------------------------------------------------------------|------------|
| 19.18     | Verify Accounting Infrastructure . . . . .                                 | 162        |
| 19.19     | Verify Persistent Job Accounting . . . . .                                 | 162        |
| 19.20     | IMPORTANT MPI Accounting Observation . . . . .                             | 163        |
| 19.21     | Current Cluster Capability . . . . .                                       | 163        |
| <b>20</b> | <b>QoS + User Limits + Scheduling Policies</b>                             | <b>163</b> |
| 20.1      | IMPORTANT CONCEPT . . . . .                                                | 164        |
| 20.2      | IMPORTANT Slurm Scheduling Hierarchy . . . . .                             | 165        |
| 20.3      | Existing Stable Accounting Infrastructure . . . . .                        | 166        |
| 20.4      | Understanding QoS . . . . .                                                | 166        |
| 20.5      | IMPORTANT QoS Concept . . . . .                                            | 166        |
| 20.6      | Verify Cluster Registration . . . . .                                      | 167        |
| 20.7      | Create Slurm Account . . . . .                                             | 168        |
| 20.8      | IMPORTANT Account Concept . . . . .                                        | 168        |
| 20.9      | Add Users to Slurm Accounting . . . . .                                    | 169        |
| 20.10     | Create QoS Policy . . . . .                                                | 169        |
| 20.11     | Configure QoS Limits . . . . .                                             | 170        |
| 20.12     | Attach QoS to Account . . . . .                                            | 171        |
| 20.13     | Verify Associations . . . . .                                              | 171        |
| 20.14     | IMPORTANT ISSUE FACED . . . . .                                            | 172        |
| 20.15     | Root Cause of QoS Enforcement Failure . . . . .                            | 173        |
| 20.16     | Enable QoS Enforcement . . . . .                                           | 173        |
| 20.17     | IMPORTANT Enforcement Concept . . . . .                                    | 174        |
| 20.18     | Distribute Updated Configuration . . . . .                                 | 174        |
| 20.19     | Restart Slurm Services . . . . .                                           | 175        |
| 20.20     | Verify Cluster Health . . . . .                                            | 175        |
| 20.21     | Verify QoS Enforcement . . . . .                                           | 175        |
| 20.22     | Verify Valid CPU Allocation . . . . .                                      | 177        |
| 20.23     | IMPORTANT MPI + QoS Integration . . . . .                                  | 177        |
| 20.24     | Verify Waltime Enforcement . . . . .                                       | 178        |
| 20.25     | IMPORTANT Pending Queue Concept . . . . .                                  | 178        |
| 20.26     | Inspect QoS Configuration . . . . .                                        | 179        |
| 20.27     | IMPORTANT Scheduler Layers . . . . .                                       | 179        |
| 20.28     | Current Cluster Capability . . . . .                                       | 179        |
| <b>21</b> | <b>Validation and Verification of Advanced Slurm Configuration and QoS</b> | <b>180</b> |
| 21.1      | Cluster Validation State . . . . .                                         | 181        |
| 21.2      | Verify Cluster Health . . . . .                                            | 181        |
| 21.3      | Verify Node Resource Recognition . . . . .                                 | 182        |

|           |                                                                 |            |
|-----------|-----------------------------------------------------------------|------------|
| 21.4      | Verify Accounting Infrastructure . . . . .                      | 183        |
| 21.5      | Verify Cluster Accounting Registration . . . . .                | 183        |
| 21.6      | Verify Accounts and User Associations . . . . .                 | 184        |
| 21.7      | Verify QoS Configuration . . . . .                              | 184        |
| 21.8      | IMPORTANT Validation Issue Faced . . . . .                      | 185        |
| 21.9      | Root Cause Analysis . . . . .                                   | 185        |
| 21.10     | Fix Applied . . . . .                                           | 186        |
| 21.11     | MPI Validation Program . . . . .                                | 186        |
| 21.12     | Validation Job Script . . . . .                                 | 187        |
| 21.13     | Verify Successful MPI Execution . . . . .                       | 187        |
| 21.14     | Verify Scheduler Environment Variables . . . . .                | 188        |
| 21.15     | Verify CPU QoS Enforcement . . . . .                            | 188        |
| 21.16     | IMPORTANT CPU QoS Concept . . . . .                             | 189        |
| 21.17     | Verify Walltime Enforcement . . . . .                           | 189        |
| 21.18     | Verify Job Count Enforcement . . . . .                          | 190        |
| 21.19     | Verify Accounting Persistence . . . . .                         | 191        |
| 21.20     | IMPORTANT MPI Accounting Observation . . . . .                  | 191        |
| 21.21     | Final Validation State . . . . .                                | 191        |
| 21.22     | Validation Conclusion . . . . .                                 | 192        |
| <b>22</b> | <b>Fairshare Scheduling and Multifactor Priority Scheduling</b> | <b>193</b> |
| 22.1      | IMPORTANT Fairshare Concept . . . . .                           | 194        |
| 22.2      | IMPORTANT Difference Between QoS and Fairshare . . . . .        | 194        |
| 22.3      | Existing Stable Scheduler Infrastructure . . . . .              | 195        |
| 22.4      | IMPORTANT Scheduler Concept . . . . .                           | 196        |
| 22.5      | Enable Multifactor Priority Scheduler . . . . .                 | 196        |
| 22.6      | IMPORTANT PriorityType Concept . . . . .                        | 196        |
| 22.7      | Understanding PriorityWeightFairshare . . . . .                 | 197        |
| 22.8      | Understanding PriorityWeightAge . . . . .                       | 197        |
| 22.9      | Understanding PriorityWeightJobSize . . . . .                   | 198        |
| 22.10     | Understanding PriorityWeightPartition . . . . .                 | 198        |
| 22.11     | Configure Fairshare Decay Settings . . . . .                    | 198        |
| 22.12     | Understanding PriorityDecayHalfLife . . . . .                   | 199        |
| 22.13     | Understanding PriorityUsageResetPeriod . . . . .                | 199        |
| 22.14     | Understanding PriorityFavorSmall . . . . .                      | 199        |
| 22.15     | Understanding FairShareDampeningFactor . . . . .                | 200        |
| 22.16     | Distribute Updated Configuration . . . . .                      | 200        |
| 22.17     | Install Configuration . . . . .                                 | 200        |
| 22.18     | Restart Slurm Services . . . . .                                | 201        |
| 22.19     | IMPORTANT Issue Initially Observed . . . . .                    | 201        |

|       |                                                   |     |
|-------|---------------------------------------------------|-----|
| 22.20 | IMPORTANT sprio Concept . . . . .                 | 202 |
| 22.21 | IMPORTANT Observation About sprio . . . . .       | 203 |
| 22.22 | Verify Fairshare Accounting . . . . .             | 203 |
| 22.23 | Understanding sshare Output . . . . .             | 204 |
| 22.24 | Understanding RawUsage . . . . .                  | 204 |
| 22.25 | Understanding FairShare Value . . . . .           | 204 |
| 22.26 | IMPORTANT Fairshare Concept . . . . .             | 205 |
| 22.27 | Create Additional User Competition . . . . .      | 205 |
| 22.28 | Generate Scheduler Contention . . . . .           | 206 |
| 22.29 | IMPORTANT Understanding Pending Reasons . . . . . | 206 |
| 22.30 | Understand sprio Output . . . . .                 | 206 |
| 22.31 | Understanding PRIORITY Column . . . . .           | 207 |
| 22.32 | Understanding FAIRSHARE Column . . . . .          | 207 |
| 22.33 | IMPORTANT Fairshare Validation . . . . .          | 208 |
| 22.34 | Understanding AGE Column . . . . .                | 208 |
| 22.35 | Understanding JOBSIZE Column . . . . .            | 208 |
| 22.36 | Understanding PARTITION Column . . . . .          | 209 |
| 22.37 | IMPORTANT Final Scheduler State . . . . .         | 209 |
| 22.38 | Final Fairshare Validation State . . . . .        | 210 |
| 22.39 | Final Conclusion . . . . .                        | 210 |

## 1 Abstract

This documentation presents the complete setup and configuration of a lightweight High Performance Computing (HPC) cluster environment using Rocky Linux and VirtualBox. The guide covers practical implementation of core HPC infrastructure components including shared storage, Slurm workload management, LDAP-based centralized authentication, MPI runtime environment, and centralized software stack management using Spack.

The cluster environment supports multinode job execution, centralized software distribution, distributed MPI applications, and scheduler-aware runtime execution. The setup follows real-world HPC concepts while remaining suitable for learning, experimentation, and practical hands-on training.

## 2 Preface

This documentation was prepared with the assistance of AI tools for:

- structuring

- formatting
- workflow organization
- command documentation
- concept explanations

However, the complete HPC cluster setup, configuration, testing, and validation were performed manually by me in a fully working VirtualBox based cluster environment.

All commands, scripts, workflows, and configurations documented in this guide were tested successfully during the setup process, including:

- cluster networking
- shared storage
- NFS configuration
- Slurm workload manager
- LDAP authentication
- centralized user management
- Spack software stack
- MPI runtime environment
- multinode execution

This guide is intended for:

- students
- beginners
- Linux learners
- HPC enthusiasts
- system administrators
- researchers

who want practical hands-on experience in:

- HPC infrastructure setup
- cluster administration
- workload scheduling
- centralized authentication
- parallel programming
- MPI execution
- software stack management

The goal of this documentation is not only to provide commands, but also to explain the practical concepts behind real HPC environments in a simple and beginner-friendly manner.

### **3 About the Author**

Hi, I'm Abhishek Raj

I work in the field of High Performance Computing (HPC), Linux systems, parallel programming, and scientific computing with experience in HPC application optimization and distributed computing environments, including the PARAM series of supercomputers.

My primary areas of interest include:

- HPC clusters and infrastructure
- Linux systems and administration
- MPI, OpenMP, CUDA, and OpenACC
- scientific and research workloads
- distributed systems
- software optimization and performance tuning
- automation and reproducible environments

This project was created to gain a deeper understanding of how HPC clusters are designed and managed internally. The objective was not only to build a working cluster environment from scratch, but also to better understand the complete software and infrastructure stack behind parallel computing systems, which ultimately helps in writing and optimizing efficient HPC applications.

GitHub Profile:

<https://github.com/CISSSCO/>

LinkedIn :

<https://www.linkedin.com/in/abhi581b>

Project Repository:

<https://github.com/CISSSCO/hpccfs>

Hosted Documentation:

<https://ciSSSCO.github.io/hpccfs>

Source codes, scripts, configuration files, and examples used in this guide are available in the project repository.

For queries, suggestions, or discussions:

<https://ciscoramon.netlify.app/>

If you encounter any issue in the documentation, setup process, or configuration steps, feel free to create an issue in the GitHub repository.

## 4 Introduction

High Performance Computing (HPC) clusters are distributed computing systems designed to execute computational workloads across multiple machines simultaneously. Instead of relying on a single system, HPC clusters combine multiple compute nodes to provide higher computational capacity, scalability, and parallel execution capability.

Modern HPC environments are widely used in:

- scientific simulations
- engineering workloads
- computational chemistry
- artificial intelligence
- machine learning
- weather modeling

- large-scale data analysis

A typical HPC cluster contains several important components working together:

| Component             | Purpose                                |
|-----------------------|----------------------------------------|
| Login Node            | User access and job submission         |
| Compute Nodes         | Execute computational workloads        |
| Shared Storage        | Shared user data and software          |
| Scheduler             | Resource allocation and job management |
| MPI Runtime           | Distributed parallel execution         |
| Authentication System | Centralized user management            |

This guide focuses on building these components step-by-step in a small virtualized environment using:

- Rocky Linux
- VirtualBox
- Slurm
- OpenMPI
- LDAP
- NFS
- Spack

Although the setup is lightweight and designed for learning purposes, the overall architecture closely follows concepts used in production HPC centers and supercomputers.

Throughout this documentation, readers will gradually configure:

- cluster networking
- shared storage
- centralized authentication
- workload scheduling
- distributed software stacks

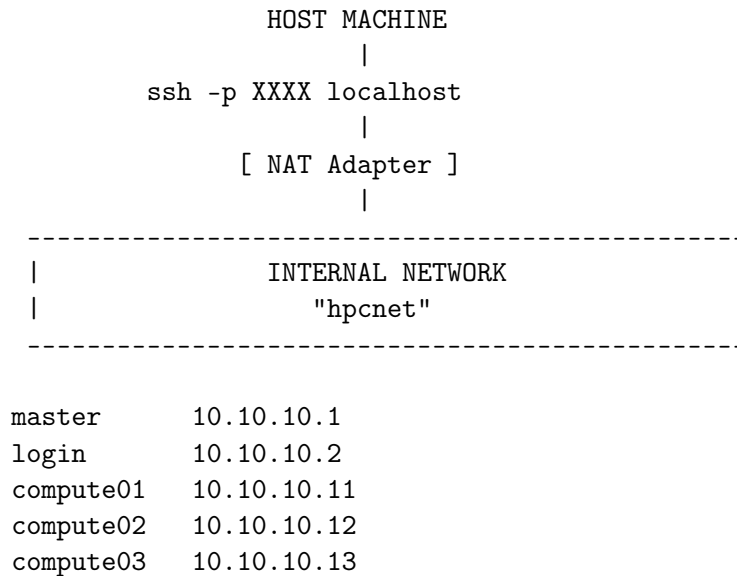


- MPI-based parallel execution
- scheduler-aware job execution

The final environment provides a fully working multinode HPC cluster capable of running distributed parallel applications across multiple compute nodes.

## 5 Cluster Architecture

The cluster consists of five virtual machines connected using an internal network.



## 6 Node Details

### 6.1 Node Role

#### 1. Master Node

The master node manages the overall cluster.

Main responsibilities:

- Slurm controller
- NFS server

- LDAP server
- Monitoring tools
- Shared software stack
- Cluster configuration

## 2. Login Node

The login node is the access point for users.

Typical tasks performed here:

- SSH login
- Code compilation
- Job submission
- Accessing shared storage

## 3. Compute Nodes

Compute nodes run the actual workloads.

These nodes are mainly used for:

- MPI jobs
- OpenMP jobs
- Batch workloads
- Parallel execution

## 6.2 Networking Setup

Two network adapters are used for each VM.

### 1. NAT Adapter

Used for:

- Internet access
- Package installation
- SSH access from host system

### 2. Internal Network

Used for communication between cluster nodes.

Network name:

`hpcnet`

This network is isolated and only used inside the cluster.

### 6.3 Static IP Addressing

Each node uses a fixed IP address.

| VM Name   | Hostname        | Internal IP | SSH Port |
|-----------|-----------------|-------------|----------|
| master    | master.local    | 10.10.10.1  | 2221     |
| login     | login.local     | 10.10.10.2  | 2222     |
| compute01 | compute01.local | 10.10.10.11 | 2223     |
| compute02 | compute02.local | 10.10.10.12 | 2224     |
| compute03 | compute03.local | 10.10.10.13 | 2225     |

Example SSH access:

```
ssh -p 2221 hpcuser@localhost
```

## 7 Resource Allocation

The following configuration is enough for a lightweight HPC lab. You can increase resources later depending on your host system.

### 7.1 Master Node

| Resource | Value |
|----------|-------|
| RAM      | 2 GB  |
| CPU      | 2     |
| Disk     | 30 GB |

### 7.2 Login Node

| Resource | Value |
|----------|-------|
| RAM      | 2 GB  |
| CPU      | 2     |
| Disk     | 25 GB |

### 7.3 Compute Nodes

| Resource | Value |
|----------|-------|
| RAM      | 2 GB  |
| CPU      | 2     |
| Disk     | 20 GB |

## 8 Master Node Setup

This section covers the complete setup of the master node. The master node acts as the central management node of the cluster.

Main responsibilities:

- Cluster management
- Shared storage
- Slurm controller
- Monitoring services
- User authentication
- Software management

### 8.1 Install VirtualBox on Manjaro

I'm using Manjaro (Arch) as my base system.

1. Update packages

```
sudo pacman -Syu
```

2. Install VirtualBox and host modules

```
sudo pacman -S virtualbox virtualbox-host-modules-arch
```

3. Install VirtualBox extension pack

```
sudo pacman -S virtualbox-ext-oracle
```

4. Load VirtualBox kernel module

```
sudo modprobe vboxdrv
```

5. Add current user to VirtualBox group

```
sudo usermod -aG vboxusers $USER
```

6. Reboot the system

```
sudo reboot
```

## 8.2 Download Rocky Linux

Download Rocky Linux Minimal ISO:

<https://rockylinux.org/download>

Choose:

- x86\_64
- Minimal ISO

The minimal installation keeps the environment lightweight and avoids unnecessary packages.

## 8.3 Create Master VM

Open VirtualBox and create a new virtual machine.

1. VM Name

`master`

2. Type

`Linux`  
`Red Hat (64-bit)`

3. Hardware

| Setting | Value   |
|---------|---------|
| RAM     | 2046 MB |
| CPU     | 2       |
| Disk    | 30 GB   |

4. Disk Type

`VDI`  
`Dynamically Allocated`

This configuration is enough for a lightweight HPC lab environment.

## 8.4 Network Configuration

### 1. Adapter 1

Attached to: NAT

Used for:

- Internet access
- Package installation
- SSH access from host machine

### 2. NAT Port Forwarding

Open:

Settings -> Network -> Adapter 1 -> Advanced -> Port Forwarding

| Name | Protocol | Host Port | Guest Port |
|------|----------|-----------|------------|
| ssh  | TCP      | 2221      | 22         |

### 3. Adapter 2

Attached to: Internal Network

Internal Network Name:

hpcnet

IMPORTANT: All nodes MUST use the same internal network name.  
This network is used for communication between cluster nodes.

## 8.5 Rocky Linux Installation

Choose:

Minimal Install

### 1. Network and Hostname

Enable BOTH network adapters.

Set hostname:

```
master.local
```

## 2. User Creation

Create an administrative user during installation.

| Field         | Value    |
|---------------|----------|
| Username      | hpcadmin |
| Administrator | YES      |

After installation, log into the VM.

## 8.6 Initial System Verification

### 1. Check Interfaces

```
ip addr
```

Expected:

- NAT interface should receive:
  - 10.0.2.15
- Internal interface should exist but may not yet have static IP

## 8.7 Configure Static Internal Network

### 1. Check Device Names

```
nmcli device status
```

Expected:

| Interface | Purpose          |
|-----------|------------------|
| enp0s3    | NAT              |
| enp0s8    | Internal Network |

## 8.8 Configure Internal Interface

Launch NetworkManager TUI:

```
sudo nmtui
```

Navigate:

```
Edit a connection
```

Select:

```
enp0s8
```

Set:

| Field       | Value         |
|-------------|---------------|
| IPv4 Method | Manual        |
| Address     | 10.10.10.1/24 |
| Gateway     | blank         |
| DNS         | blank         |

DO NOT modify NAT interface.

## 8.9 Restart Networking

```
sudo systemctl restart NetworkManager
```

## 8.10 Verify Networking

```
ip addr
```

Expected:

| Interface | IP         |
|-----------|------------|
| NAT       | 10.0.2.x   |
| Internal  | 10.10.10.1 |

## 8.11 SSH Access from Host

From host machine:

```
ssh -p 2221 hpcadmin@localhost
```

If the connection works, port forwarding is configured correctly.



## 8.12 System Update and Packages

### 1. Update System

```
sudo dnf update -y
```

### 2. Install Base Packages

```
sudo dnf install -y \  
vim wget curl git \  
net-tools bind-utils \  
htop tmux tree \  
gcc gcc-c++ gcc-gfortran \  
make cmake \  
python3 \  
openssh-server
```

These packages provide:

- Development tools
- Network utilities
- Monitoring utilities
- Basic administration tools

## 8.13 Timezone Configuration

Check timezone:

```
timedatectl
```

Set timezone if needed:

```
sudo timedatectl set-timezone Asia/Kolkata
```

## 8.14 Configure `/etc/hosts`

Edit:

```
sudo vim /etc/hosts
```

Add:

```
10.10.10.1  master master.local
10.10.10.2  login login.local
10.10.10.11 compute01 compute01.local
10.10.10.12 compute02 compute02.local
10.10.10.13 compute03 compute03.local
```

This allows hostname-based communication between nodes.

### 8.15 Disable Firewall (Temporary)

For initial setup and testing:

```
sudo systemctl disable --now firewalld
```

NOTE:

Firewall rules can be configured properly later after cluster setup is complete.

### 8.16 SELinux Configuration

Check status:

```
getenforce
```

Temporarily disable enforcing:

```
sudo setenforce 0
```

Persist configuration:

```
sudo vim /etc/selinux/config
```

Modify:

```
SELINUX=permissive
```

This helps avoid permission-related issues during initial cluster setup.

### 8.17 Verify Hostname

```
hostnamectl
```

Expected:

```
Static hostname: master.local
```

If hostname is not updated(showing previous), reboot the machine.

At this point, the master node setup is complete and ready for cloning or further cluster configuration.

## 9 Cloning Nodes

Purpose:

- avoid reinstalling all nodes manually
- simulate real HPC image provisioning workflow

### 9.1 Create Snapshot

Power off VM:

```
sudo poweroff
```

Create snapshot in VirtualBox:

```
clean-base-master
```

### 9.2 Clone Nodes

Create FULL CLONES.

IMPORTANT: Enable:

Reinitialize the MAC address of all network cards

### 9.3 Clone Names

| VM Name   |
|-----------|
| login     |
| compute01 |
| compute02 |
| compute03 |

### 9.4 Configure Port Forwarding

1. Login Node

| Host Port | Guest Port |
|-----------|------------|
| 2222      | 22         |

2. Compute01

| Host Port | Guest Port |
|-----------|------------|
| 2223      | 22         |

### 3. Compute02

| Host Port | Guest Port |
|-----------|------------|
| 2224      | 22         |

### 4. Compute03

| Host Port | Guest Port |
|-----------|------------|
| 2225      | 22         |

## 9.5 Machine ID Regeneration

IMPORTANT: Each cloned Linux machine must have unique machine-id.

## 9.6 Remove Existing Machine ID

```
sudo rm -f /etc/machine-id
```

## 9.7 Generate New Machine ID

```
sudo systemd-machine-id-setup
```

## 9.8 Verify Machine ID

```
cat /etc/machine-id
```

Ensure every node has unique machine-id.

## 9.9 Configure Hostname

### 1. Login Node

Edit hostname:

```
sudo vim /etc/hostname
```

Set:

```
login.local
```

Apply:

```
sudo hostnamectl set-hostname login.local
```

2. Compute01

Set hostname:

```
compute01.local
```

Apply:

```
sudo hostnamectl set-hostname compute01.local
```

3. Compute02

```
compute02.local
```

```
sudo hostnamectl set-hostname compute02.local
```

4. Compute03

```
compute03.local
```

```
sudo hostnamectl set-hostname compute03.local
```

## 9.10 Configure Static Internal IPs

Launch:

```
sudo nmtui
```

Edit internal adapter:

```
enp0s8
```

1. Login Node

```
10.10.10.2/24
```

2. Compute01

10.10.10.11/24

3. Compute02

10.10.10.12/24

4. Compute03

10.10.10.13/24

## 9.11 Restart Networking

```
sudo systemctl restart NetworkManager
```

## 9.12 Verify Networking

```
ip addr
```

## 9.13 Verify Routing

```
ip route
```

Expected:

```
default via 10.0.2.2 dev enp0s3  
10.10.10.0/24 dev enp0s8
```

## 9.14 Verify Connectivity

1. Ping Between Nodes

```
ping master
```

```
ping login
```

```
ping compute01
```

## 9.15 SSH Verification

From host:

1. Master

```
ssh -p 2221 hpcadmin@localhost
```

2. Login

```
ssh -p 2222 hpcadmin@localhost
```

3. Compute01

```
ssh -p 2223 hpcadmin@localhost
```

4. Compute02

```
ssh -p 2224 hpcadmin@localhost
```

5. Compute03

```
ssh -p 2225 hpcadmin@localhost
```

## 10 Shared Infrastructure Services

Goal:

Transform the cluster into a realistic HPC-style shared environment.

This phase includes:

- passwordless SSH
- shared home directories
- NFS server/client configuration
- persistent mounts
- multi-node shared filesystem verification

By the end of this phase:

- all nodes share same /home
- users can access same files from all nodes
- cluster behaves like a real HPC environment

## 10.1 Cluster Role Architecture

### 1. Master Node

Purpose:

- cluster management
- NFS server
- future Slurm controller
- future Munge authority
- future LDAP server

Services:

| Service       | Purpose              |
|---------------|----------------------|
| nfs-server    | shared storage       |
| future slurmd | scheduler controller |
| future munge  | authentication       |

### 2. Login Node

Purpose:

- user login
- compilation
- job submission

Users should:

- SSH into login node
- compile programs
- submit jobs using Slurm

### 3. Compute Nodes

Purpose:

- execute jobs only

Future services:

| Service | Purpose        |
|---------|----------------|
| slurmd  | compute daemon |
| munge   | authentication |



## 10.2 Verify User Consistency

IMPORTANT:

NFS shared storage requires same UID/GID across all nodes.

Verify user identity on all nodes:

```
id hpcadmin
```

Expected:

```
uid=1000(hpcadmin) gid=1000(hpcadmin)
```

All nodes must match.

## 10.3 Passwordless SSH Configuration

Purpose:

- MPI communication
- Slurm execution
- cluster automation
- remote management

SSH trust is generated ONLY from login node.

This mimics real HPC architecture:

User -> Login Node -> Compute Nodes

## 10.4 Generate SSH Key on Login Node

SSH into login node from host:

```
ssh -p 2222 hpcadmin@localhost
```

Generate SSH key:

```
ssh-keygen
```

Press Enter for all prompts.

Generated files:

```
~/.ssh/id_rsa
```

```
~/.ssh/id_rsa.pub
```

## 10.5 Copy SSH Keys to Cluster Nodes

1. Master

```
ssh-copy-id hpcadmin@master
```

2. Compute01

```
ssh-copy-id hpcadmin@compute01
```

3. Compute02

```
ssh-copy-id hpcadmin@compute02
```

4. Compute03

```
ssh-copy-id hpcadmin@compute03
```

## 10.6 Verify Passwordless SSH

1. Test Master

```
ssh master hostname
```

Expected:

```
master.local
```

2. Test Compute01

```
ssh compute01 hostname
```

Expected:

```
compute01.local
```

3. Test Compute02

```
ssh compute02 hostname
```

4. Test Compute03

```
ssh compute03 hostname
```

No password prompts should appear.

## 10.7 HPC Concept — Why Passwordless SSH Matters

Passwordless SSH becomes essential for:

| Feature    | Purpose               |
|------------|-----------------------|
| MPI        | remote process launch |
| Slurm      | distributed execution |
| automation | orchestration         |
| monitoring | remote administration |

## 10.8 NFS Shared Storage Setup

Purpose:

Provide shared home directories across all cluster nodes.

This is a core HPC architecture requirement.

Without shared storage:

- source code differs per node
- binaries differ
- scripts unavailable across nodes
- MPI execution becomes difficult

Real HPC systems commonly use:

- NFS
- Lustre
- BeeGFS
- GPFS

This lab begins with NFS.

## 10.9 Install NFS Packages

Install on ALL nodes:

```
sudo dnf install -y nfs-utils
```

## 10.10 Configure NFS Server on Master

SSH into master node.

## 10.11 Enable NFS Services

```
sudo systemctl enable --now nfs-server
```

Verify:

```
systemctl status nfs-server
```

Expected:

```
active (running)
```

## 10.12 Configure NFS Exports

Edit exports file:

```
sudo vim /etc/exports
```

Add:

```
/home 10.10.10.0/24(rw,sync,no_root_squash)
```

## 10.13 Export Option Explanation

| Option         | Meaning                  |
|----------------|--------------------------|
| rw             | read/write access        |
| sync           | synchronous writes       |
| no_root_squash | preserve root privileges |

## 10.14 Apply Exports

```
sudo exportfs -rav
```

## 10.15 Verify Exports

```
sudo exportfs -v
```

Expected export:

```
/home 10.10.10.0/24(...)
```

## 10.16 Configure NFS Clients

Perform on:

- login
- compute01
- compute02
- compute03

## 10.17 Backup Existing Home Directory

```
sudo mv /home /home.local
```

Purpose:

Preserve original local home directory.

## 10.18 Create New Mount Point

```
sudo mkdir /home
```

## 10.19 Mount Shared NFS Home

```
sudo mount master:/home /home
```

## 10.20 Verify Mount

```
df -h
```

Expected:

```
master:/home
```

mounted on:

```
/home
```

## 10.21 Verify Shared Filesystem

From login node:

```
touch ~/shared_test
```

From compute01:

```
ls ~
```

Expected:

```
shared_test
```

This confirms successful shared filesystem operation.

## 10.22 Configure Persistent NFS Mounts

Without this configuration:

- NFS mount disappears after reboot

## 10.23 Configure `/etc/fstab`

On ALL client nodes:

```
sudo vim /etc/fstab
```

Add:

```
master:/home /home nfs defaults 0 0
```

## 10.24 Verify `fstab` Before Reboot

IMPORTANT:

Always validate `fstab` before rebooting.

Run:

```
sudo mount -a
```

If no errors appear:

- `fstab` configuration is correct

## 10.25 Reboot Validation Procedure

Purpose:

Validate persistent cluster infrastructure after reboot.

This is important real-world HPC administration practice.

## 10.26 Reboot Order

1. Reboot Master First

```
sudo reboot
```

Wait until:

- SSH becomes available
- NFS server active

Verify:

```
systemctl status nfs-server
```

2. Reboot Login Node

```
sudo reboot
```

3. Reboot Compute Nodes

```
sudo reboot
```

Perform for:

- compute01
- compute02
- compute03

## 10.27 Verify Persistent Mounts After Reboot

On login node:

```
df -h
```

Expected:

```
master:/home
```

mounted automatically.

## 10.28 Verify Active NFS Mount

```
mount | grep home
```

Expected:

```
master:/home on /home type nfs
```

## 10.29 Verify Shared Files After Reboot

```
ls ~
```

Expected:

- previously created shared files still visible

# 11 Munge Authentication + Slurm Scheduler

Goal:

Transform the cluster into a scheduler-managed HPC environment.

This phase includes:

- Munge authentication
- Slurm controller setup
- Compute node daemon setup
- Scheduler configuration
- Partition creation
- Multi-node execution
- Resource allocation testing

By the end of this phase:

- cluster nodes authenticate securely
- Slurm manages compute resources
- jobs execute across nodes
- scheduler topology becomes operational



## 11.1 HPC Scheduler Architecture

### 1. Master Node

Purpose:

- Slurm controller
- cluster scheduler
- resource manager
- Munge authority

Services:

| Service   | Purpose                |
|-----------|------------------------|
| munged    | cluster authentication |
| slurmctld | scheduler controller   |

### 2. Login Node

Purpose:

- user login
- compilation
- job submission

Users should:

- login here
- compile here
- submit jobs here

### 3. Compute Nodes

Purpose:

- execute jobs

Services:

| Service | Purpose                  |
|---------|--------------------------|
| munged  | authentication           |
| slurmd  | compute execution daemon |

## 11.2 IMPORTANT Rocky Linux 10 Issue

Originally the cluster was created using:

Rocky Linux 10

Problem encountered:

No match for argument: slurm

Reason:

- Rocky Linux 10 HPC ecosystem still immature
- Slurm packages unavailable in standard repos
- OpenHPC EL10 support incomplete

Resolution:

- cluster rebuilt using Rocky Linux 9 Minimal

Important HPC lesson:

Production HPC clusters usually avoid bleeding-edge enterprise releases because:

- scheduler ecosystems lag behind
- HPC repos may not exist
- compatibility becomes unstable

## 11.3 Install EPEL Repository

Install on ALL nodes:

```
sudo dnf install -y epel-release
```

## 11.4 Enable CRB Repository

Required on Rocky Linux 9 and 10.

Install on ALL nodes:

```
sudo crb enable
```

## 11.5 Install Munge + Slurm Packages

Install on ALL nodes:

```
sudo dnf install -y \  
munge munge-libs \  
slurm slurm-slurmd slurm-slurmctld
```

## 11.6 Verify Installed Packages

1. Verify Munge

```
rpm -qa | grep munge
```

Expected:

```
munge  
munge-libs
```

2. Verify Slurm

```
rpm -qa | grep slurm
```

Expected:

```
slurm  
slurm-slurmd  
slurm-slurmctld
```

## 11.7 Verify System Users

1. Verify Munge User

```
id munge
```

2. Verify Slurm User

```
id slurm
```

## 11.8 IMPORTANT Issue Encountered — Missing slurm User

Problem:

```
id: 'slurm': no such user
```

Reason:

Some Slurm packages on Rocky Linux do not automatically create the slurm user.

Resolution:

Create manually on ALL nodes.

## 11.9 Create Slurm Group

```
sudo groupadd slurm
```

## 11.10 Create Slurm User

```
sudo useradd -r -g slurm -s /sbin/nologin slurm
```

Explanation:

| Option           | Meaning             |
|------------------|---------------------|
| -r               | system account      |
| -g slurm         | primary group       |
| -s /sbin/nologin | disable shell login |

## 11.11 Verify Slurm User

```
id slurm
```

Expected:

```
uid=...
```

```
gid=...
```

## 11.12 IMPORTANT HPC Security Concept

Cluster daemons should not run permanently as root.

Therefore:

- Munge runs as munge user
- Slurm runs as slurm user

This is standard enterprise daemon security practice.

### 11.13 Configure Munge Authentication

Purpose:

Create secure trust relationship between all cluster nodes.

IMPORTANT:

All nodes MUST use the same:

`/etc/munge/munge.key`

This key acts as the cluster-wide shared secret.

### 11.14 Generate Munge Key on Master

ONLY on master node:

```
sudo /usr/sbin/create-munge-key
```

### 11.15 Verify Munge Key

```
ls -l /etc/munge/munge.key
```

Expected:

```
-rw-----
```

owned by:

```
munge munge
```

### 11.16 IMPORTANT Mistake Encountered — Munge Key Generated on All Nodes

Problem:

Munge key accidentally generated independently on all nodes.

Result:

Nodes could not authenticate each other.

Important HPC lesson:

Munge works using a:

`shared trust model`

All nodes MUST share identical munge.key.

### 11.17 Correct Munge Key Distribution

1. Copy Key Temporarily

On master:

```
sudo cp /etc/munge/munge.key /tmp/
```

2. Temporary Permission Adjustment

```
sudo chmod 644 /tmp/munge.key
```

Purpose:

Allow normal user SCP transfer.

### 11.18 IMPORTANT SSH Root Login Issue

Problem encountered:

Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password)

Reason:

Using:

```
sudo scp ...
```

attempted root SSH login.

Rocky Linux disables direct root SSH login by default.

Resolution:

Use normal user SCP.

### 11.19 Copy Munge Key to Login Node

```
scp /tmp/munge.key login:/tmp/
```

### 11.20 Copy Munge Key to Compute01

```
scp /tmp/munge.key compute01:/tmp/
```

### 11.21 Copy Munge Key to Compute02

```
scp /tmp/munge.key compute02:/tmp/
```

### 11.22 Copy Munge Key to Compute03

```
scp /tmp/munge.key compute03:/tmp/
```

### 11.23 Remove Temporary Key Copy

On master:

```
sudo rm -f /tmp/munge.key
```

### 11.24 Install Munge Key on Client Nodes

Perform on:

- login
- compute01
- compute02
- compute03

### 11.25 Stop Munge Service

```
sudo systemctl stop munge
```

### 11.26 Install Key

```
sudo mv /tmp/munge.key /etc/munge/
```

### 11.27 Fix Ownership

```
sudo chown munge:munge /etc/munge/munge.key
```

### 11.28 Fix Permissions

```
sudo chmod 400 /etc/munge/munge.key
```

### 11.29 Start Munge

On ALL nodes:

```
sudo systemctl enable --now munge
```

### 11.30 IMPORTANT Issue Encountered — Munge Not Running on Master

Problem:

```
munge: Error: Failed to access "/var/run/munge/munge.socket.2"
```

Reason:

Munge service never started on master.

Resolution:

```
sudo systemctl enable --now munge
```

Important lesson:

Difference between:

| State    | Meaning             |
|----------|---------------------|
| inactive | service not started |
| failed   | service crashed     |

### 11.31 Verify Munge Service

```
systemctl status munge
```

Expected:

```
active (running)
```

### 11.32 Verify Munge Authentication

From master:

```
munge -n | ssh compute01 unmunge
```

### 11.33 Test Compute02

```
munge -n | ssh compute02 unmunge
```

### 11.34 Test Compute03

```
munge -n | ssh compute03 unmunge
```



### 11.35 Test Login Node

```
munge -n | ssh login unmunge
```

Expected:

STATUS: Success

### 11.36 HPC Concepts Learned

Munge introduced:

- cluster-wide authentication
- distributed trust model
- shared secret authentication
- daemon security
- inter-node trust validation

### 11.37 Configure Slurm Scheduler

Purpose:

Enable centralized resource scheduling.

IMPORTANT:

```
/etc/slurm/slurm.conf
```

must be IDENTICAL on ALL nodes.

### 11.38 IMPORTANT Issue Encountered — Default localhost Configuration

Problem:

```
sinfo
```

```
-> localhost
```

Reason:

Default example slurm.conf remained active.

Resolution:

Replace configuration with custom cluster topology.

Important HPC lesson:

Incorrect topology configuration is one of the most common Slurm admin problems.

## 11.39 Create Slurm Configuration

On master:

```
sudo vim /etc/slurm/slurm.conf
```

Replace ENTIRE file with:

```
ClusterName=hpccluster
```

```
SlurmctldHost=master
```

```
MpiDefault=none
```

```
ProctrackType=proctrack/cgroup
```

```
ReturnToService=2
```

```
SlurmctldPidFile=/run/slurmctld.pid
```

```
SlurmdPidFile=/run/slurmd.pid
```

```
SlurmdSpoolDir=/var/spool/slurmd
```

```
StateSaveLocation=/var/spool/slurmctld
```

```
SlurmUser=slurm
```

```
SwitchType=switch/none
```

```
TaskPlugin=task/affinity
```

```
SchedulerType=sched/backfill
```

```
SelectType=select/cons_tres
```

```
SelectTypeParameters=CR_Core
```

```
AuthType=auth/munge
```

```
NodeName=compute01 CPUs=2 State=UNKNOWN
```

```
NodeName=compute02 CPUs=2 State=UNKNOWN
```

```
NodeName=compute03 CPUs=2 State=UNKNOWN
```

```
PartitionName=debug Nodes=ALL Default=YES MaxTime=INFINITE State=UP
```

## 11.40 Configuration Explanation

| Parameter     | Purpose            |
|---------------|--------------------|
| ClusterName   | cluster identifier |
| SlurmctldHost | controller node    |
| NodeName      | compute nodes      |
| CPUs=2        | available CPUs     |
| PartitionName | scheduler queue    |

## 11.41 Create Slurm Runtime Directories

1. On Master

```
sudo mkdir -p /var/spool/slurmctld
```

```
sudo mkdir -p /var/spool/slurmd
```

## 11.42 Set Ownership

```
sudo chown slurm:slurm /var/spool/slurmctld
```

```
sudo chown slurm:slurm /var/spool/slurmd
```

## 11.43 Create slurmd Directories on Compute Nodes

1. Compute01

```
ssh compute01 sudo -S mkdir -p /var/spool/slurmd
```

2. Compute02

```
ssh compute02 sudo -S mkdir -p /var/spool/slurmd
```

3. Compute03

```
ssh compute03 sudo -S mkdir -p /var/spool/slurmd
```

## 11.44 IMPORTANT sudo Over SSH Issue

Problem:

```
sudo: a terminal is required to read the password
```

Resolution:

Use:

```
sudo -S
```

Important Linux administration concept:

sudo over non-interactive SSH requires stdin password handling.

## 11.45 Fix Ownership on Compute Nodes

1. Compute01

```
ssh compute01 sudo -S chown slurm:slurm /var/spool/slurmd
```

2. Compute02

```
ssh compute02 sudo -S chown slurm:slurm /var/spool/slurmd
```

3. Compute03

```
ssh compute03 sudo -S chown slurm:slurm /var/spool/slurmd
```

## 11.46 Verify Runtime Directories

```
ls -ld /var/spool/slurmctld
```

Expected:

```
slurm slurm
```

### 11.47 Distribute `slurm.conf`

1. Login

```
scp /etc/slurm/slurm.conf login:/tmp/
```

2. Compute01

```
scp /etc/slurm/slurm.conf compute01:/tmp/
```

3. Compute02

```
scp /etc/slurm/slurm.conf compute02:/tmp/
```

4. Compute03

```
scp /etc/slurm/slurm.conf compute03:/tmp/
```

### 11.48 Install Configuration on Nodes

On ALL nodes:

```
sudo mv /tmp/slurm.conf /etc/slurm/slurm.conf
```

### 11.49 Start Slurm Controller

ONLY on master:

```
sudo systemctl enable --now slurmctld
```

### 11.50 Verify Controller

```
systemctl status slurmctld
```

Expected:

active (running)

### 11.51 Start slurmd on Compute Nodes

On:

- compute01
- compute02
- compute03

```
sudo systemctl enable --now slurmd
```

### 11.52 Verify slurmd

```
systemctl status slurmd
```

Expected:

active (running)

### 11.53 Reconfigure Slurm

On master:

```
sudo scontrol reconfigure
```

### 11.54 IMPORTANT Issue Encountered — Nodes in DRAIN State

Problem:

Nodes entered:

DRAIN

after CPU topology changes.

Reason:

Slurm detected mismatch between:

- configured CPUs
- detected hardware topology

Resolution:

Clear stale slurmd state.

### 11.55 Stop slurmd

On compute nodes:

```
sudo systemctl stop slurmd
```

### 11.56 Remove Old State

```
sudo rm -rf /var/spool/slurmd/*
```

### 11.57 Restart slurmd

```
sudo systemctl start slurmd
```

### 11.58 Resume Nodes

On master:

```
sudo scontrol update NodeName=compute01 State=RESUME
```

```
sudo scontrol update NodeName=compute02 State=RESUME
```

```
sudo scontrol update NodeName=compute03 State=RESUME
```

### 11.59 Verify Cluster Status

```
sinfo
```

Expected:

```
PARTITION AVAIL TIMELIMIT NODES STATE NODELIST
debug* up infinite 3 idle compute[01-03]
```

### 11.60 Verify Node Details

```
scontrol show nodes
```

This displays:

- CPU information
- memory
- node states
- scheduler topology
- resource allocation

### 11.61 Test Slurm Execution

#### 1. Single Node Test

From login node:

```
srun hostname
```

Expected:

```
compute01.local
```

### 11.62 Multi-node Execution

```
srun -N 3 hostname
```

Expected:

```
compute01.local
```

```
compute02.local
```

```
compute03.local
```

### 11.63 Multi-task Multi-node Execution

```
srun -N 3 -n 6 hostname
```

This verifies:

- task distribution
- multi-node scheduling
- CPU allocation

### 11.64 Multi-core Single-node Execution

```
srun -N 1 -n 2 hostname
```

Expected:

```
compute01.local
```

```
compute01.local
```



## 11.65 IMPORTANT Slurm Scheduling Concept

Earlier:

```
srun -N 1 -n 2 hostname
```

failed because nodes initially had:

```
CPUs=1
```

After increasing VM CPUs and updating slurm.conf:

```
CPUs=2
```

the scheduler successfully allocated 2 tasks on a single node.

Important HPC lesson:

Slurm strictly enforces:

- CPU limits
- task allocation
- topology constraints
- resource consistency

## 12 Shared Software Filesystem (/apps)

Goal:

Create a dedicated shared software filesystem for:

- Spack
- compilers
- MPI stacks
- scientific applications
- modulefiles
- HPC software hierarchy

This phase transitions the cluster from:

```
basic scheduler infrastructure
```

to:

```
realistic HPC software environment
```

## 12.1 Why Separate /apps Filesystem?

Initially:

- each VM had a 30 GB local disk
- /home was exported from master over NFS

Concern encountered:

Shared /home only had ~30 GB available.

Spack and HPC software stacks can become very large.

Important HPC storage concept:

Real HPC clusters usually separate:

| Filesystem | Purpose                |
|------------|------------------------|
| /home      | user files             |
| /apps      | software stack         |
| /scratch   | temporary runtime data |
| /archive   | long-term storage      |

Therefore a dedicated:

/apps

filesystem was created.

This is significantly closer to real HPC architecture.

## 12.2 IMPORTANT Storage Architecture Concept

Each VM still has its own independent local storage.

Example:

| Node      | Local Disk |
|-----------|------------|
| master    | 30 GB      |
| login     | 30 GB      |
| compute01 | 30 GB      |
| compute02 | 30 GB      |
| compute03 | 30 GB      |

When:

master:/home

was mounted via NFS:

- compute node local /home became hidden
- but local disks still exist

Only the exported filesystem becomes shared.  
Therefore:

/apps

was created as a dedicated shared software layer.

### 12.3 Add New Virtual Disk to Master

A second virtual disk was attached ONLY to:

master

Purpose:

- centralized software storage
- shared application filesystem
- Spack software stack

Disk size selected:

60 GB

Disk type:

VDI (Dynamically Allocated)

## 12.4 Verify New Disk Detection

After booting master:

```
lsblk
```

Expected:

```
sda    30G
sdb    60G
```

Important:

| Device | Purpose       |
|--------|---------------|
| sda    | OS disk       |
| sdb    | New apps disk |

Only:

```
sdb
```

was modified.

## 12.5 Create GPT Partition Table

```
sudo fdisk /dev/sdb
```

Inside fdisk:

Create GPT label:

```
g
```

Create partition:

```
n
```

Accept defaults.

Write changes:

```
w
```

## 12.6 Verify Partition

```
lsblk
```

Expected:

```
sdb
sdb1
```

## 12.7 Create XFS Filesystem

```
sudo mkfs.xfs /dev/sdb1
```

Why XFS?

- default enterprise filesystem
- highly scalable
- excellent for large filesystems
- commonly used in HPC

## 12.8 Create Mount Point

```
sudo mkdir -p /apps
```

## 12.9 Get Filesystem UUID

```
sudo blkid /dev/sdb1
```

Example:

```
UUID="xxxxxxxx-xxxx-xxxx"
```

## 12.10 Configure Persistent Mount

Edit:

```
sudo vim /etc/fstab
```

Add:

```
UUID=YOUR_UUID_HERE    /apps    xfs    defaults    0 0
```

Important Linux administration concept:

UUIDs are preferred over:

```
/dev/sdb1
```

because device names can change after reboot.

### 12.11 Mount Filesystem

```
sudo mount -a
```

### 12.12 Verify Mount

```
df -h
```

Expected:

```
/apps
```

mounted with approximately:

```
60 GB
```

### 12.13 Configure Ownership

```
sudo chown hpcadmin:hpcadmin /apps
```

### 12.14 Create Shared HPC Software Hierarchy

```
mkdir -p /apps/{spack,modulefiles,software,sources}
```

Resulting structure:

```
/apps
  spack
  modulefiles
  software
  sources
```

## 12.15 HPC Software Hierarchy Concept

Purpose of directories:

| Directory   | Purpose               |
|-------------|-----------------------|
| spack       | Spack package manager |
| modulefiles | custom modulefiles    |
| software    | installed software    |
| sources     | downloaded sources    |

This is similar to production HPC software layouts.

## 12.16 Export /apps via NFS

Edit exports file:

```
sudo vim /etc/exports
```

Add:

```
/apps 10.10.10.0/24(rw,sync,no_root_squash)
```

## 12.17 Apply NFS Exports

```
sudo exportfs -rav
```

## 12.18 Verify Exports

```
showmount -e localhost
```

Expected:

```
/apps
```

```
/home
```

## 12.19 Create Mount Points on Client Nodes

On:

- login
- compute01
- compute02

- compute03

Create:

```
sudo mkdir -p /apps
```

## 12.20 Configure Persistent NFS Mounts

Edit:

```
sudo vim /etc/fstab
```

Add:

```
master:/apps /apps nfs defaults,_netdev 0 0
```

## 12.21 Mount Shared /apps Filesystem

```
sudo mount -a
```

## 12.22 Verify Shared Mount

```
df -h
```

Expected:

```
master:/apps
```

mounted on all nodes.

## 12.23 Verify Shared Filesystem Functionality

On master:

```
touch /apps/testfile
```

On compute node:

```
ls /apps
```

Expected:

```
testfile
```

This verifies:

- shared software filesystem
- NFS functionality
- cluster-wide software visibility



## 12.24 HPC Architecture After /apps Setup

Current cluster storage architecture:

| Filesystem  | Purpose               |
|-------------|-----------------------|
| /home       | user files            |
| /apps       | software stack        |
| local disks | local runtime storage |

This resembles real HPC environments significantly more closely.

## 13 Spack Software Stack + MPI Runtime Validation

Goal:

Create centralized HPC software stack using:

- Spack
- shared software filesystem
- shared compilers
- shared MPI stack
- scheduler-aware runtime execution

This phase validates:

- software distribution
- multinode execution
- Slurm integration
- MPI runtime functionality

After completing this section you will be able to:

- install HPC software using Spack
- manage multiple compiler versions
- manage multiple MPI versions

- load software dynamically
- compile MPI applications
- execute distributed jobs using Slurm
- monitor and debug jobs
- understand performance metrics

### 13.1 IMPORTANT HPC Software Stack Concept

Instead of:

```
dnf install openmpi
```

a centralized HPC software stack was created.

Benefits:

- software installed once
- accessible cluster-wide
- compiler hierarchy support
- reproducible builds
- module-based software environments
- centralized software management

This approach is significantly closer to real HPC centers.

In production HPC environments:

- operating system packages are separated
- scientific software stack is separated
- compiler stacks are separated
- MPI stacks are separated

This separation improves:

- portability
- reproducibility
- scalability
- maintenance

## 13.2 IMPORTANT Shared Filesystem Concept

The cluster uses:

`/apps`

as shared application storage.

Because:

`/apps`

is mounted cluster-wide through NFS:

- software installed once becomes visible everywhere
- compute nodes automatically access installed software
- users share common software stack
- recompilation is unnecessary on compute nodes

This is one of the MOST important HPC architecture concepts.

## 13.3 Install Development Tools

Installed manually on ALL nodes:

```
sudo dnf groupinstall -y "Development Tools"
```

Purpose:

Provides:

- gcc
- g++
- gfortran
- make
- linker tools
- build utilities
- development environment

Important note:

System compilers are still required even when using Spack.

Reason:

Spack builds software from source code and therefore still requires:

- assembler
- linker
- basic compiler toolchain

### 13.4 Move into Shared Application Filesystem

Run on:

`master`

Move into shared applications directory:

`cd /apps`

Verify:

`pwd`

Expected:

`/apps`

### 13.5 Clone Spack into Shared Filesystem

Clone Spack:

`git clone https://github.com/spack/spack.git`

Expected:

`/apps/spack`

This creates centralized package manager installation.

## 13.6 Enter Spack Directory

```
cd spack
```

Verify:

```
pwd
```

Expected:

```
/apps/spack
```

## 13.7 Initialize Spack Environment

Initialize Spack:

```
source share/spack/setup-env.sh
```

Purpose:

Adds:

- spack command
- shell integration
- environment variables
- package management environment

to current shell session.

Without initialization:

- spack command will not work
- spack load will fail
- software environments will not activate

## 13.8 IMPORTANT Shell Environment Concept

`source`

does NOT execute script normally.  
Instead:

it modifies current shell environment

This is extremely important in:

- Spack
- modules
- MPI environments
- compiler environments

## 13.9 Make Spack Permanent

To avoid sourcing manually every session:  
Edit:

`vim ~/.bashrc`

Add:

`source /apps/spack/share/spack/setup-env.sh`

Reload shell:

`source ~/.bashrc`

Verify:

`spack --version`

Expected:

- Spack version information

### 13.10 Clone Additional Spack Repository

Clone repository:

```
git clone https://github.com/spack/spack-packages.git
```

Purpose:

Adds additional package definitions.

### 13.11 Configure Spack Repository

```
spack repo set --destination "$(pwd)/spack-packages" builtin
```

Purpose:

Configure additional repository support.

This allows:

- package metadata
- package recipes
- software definitions

to become visible to Spack.

### 13.12 Verify Spack Functionality

```
spack info gcc
```

Expected:

Detailed GCC package information.

This verifies:

- Spack environment working
- package repository functional
- package metadata accessible

### 13.13 IMPORTANT HPC Software Stack Concept

Spack operates as:

source-based HPC package manager

Unlike:

| Package Manager | Purpose                |
|-----------------|------------------------|
| dnf             | operating system       |
| Spack           | HPC software ecosystem |

Production HPC systems commonly separate:

- OS packages
- scientific software stacks

exactly like this setup.

### 13.14 IMPORTANT Spack Build Concept

When installing software using Spack:

software is compiled from source

Advantages:

- architecture optimization
- compiler flexibility
- dependency control
- reproducibility
- variant support

Disadvantages:

- longer installation time



### 13.15 Detect Available Compilers

Before installing software:

```
spack compiler find
```

Purpose:

Detect system compilers available for building software.

Verify detected compilers:

```
spack compilers
```

Expected:

- compiler list

### 13.16 Install GCC Using Spack

Install GCC version:

```
spack install gcc@13.4.0
```

Purpose:

Create dedicated HPC compiler stack.

This may take considerable time because:

- GCC itself is compiled from source
- dependencies are built
- optimization occurs

### 13.17 IMPORTANT Multiple Compiler Version Concept

Production HPC systems frequently maintain multiple compiler versions simultaneously.

Example:

```
gcc@11  
gcc@12  
gcc@13  
intel  
nvhpc  
clang
```

Reasons:

- application compatibility
- compiler ABI differences
- reproducibility
- benchmark consistency

### 13.18 Verify Installed Software

List installed packages:

```
spack find
```

Expected example:

```
==> 1 installed package  
gcc@13.4.0
```

### 13.19 Load Installed GCC Compiler

```
spack load gcc@13.4.0
```

Purpose:

Activate compiler environment.

Verify compiler path:

```
which gcc
```

Expected:

- Spack GCC path

Verify version:

```
gcc --version
```

Expected:

- GCC 13.4.0

## 13.20 IMPORTANT Runtime Environment Concept

When software is loaded:

- PATH changes
- LD\_LIBRARY\_PATH changes
- MANPATH changes

This allows applications to locate:

- executables
- shared libraries
- runtime dependencies

automatically.

## 13.21 Register Newly Installed Compiler

After installing GCC:

```
spack compiler find
```

Purpose:

Allow Spack to use newly installed GCC for future package builds.

Verify:

```
spack compilers
```

Expected:

- new GCC compiler entry

## 13.22 Install OpenMPI Using Spack

Install MPI runtime:

```
spack install openmpi
```

Purpose:

Create shared MPI runtime stack.

This installation includes:

- MPI runtime
- MPI compiler wrappers
- MPI libraries
- distributed runtime environment

### 13.23 IMPORTANT MPI Compiler Dependency Concept

MPI stacks are tightly coupled with compiler toolchains.

Example:

`openmpi` built using `gcc@13`

This is important because:

- ABI compatibility matters
- runtime compatibility matters
- mixed compiler environments may fail

### 13.24 Verify Installed Software Tree

Show dependency tree:

`spack find -d`

Purpose:

Displays:

- dependency hierarchy
- compiler hierarchy
- runtime dependencies

Example:

```
openmpi
^gcc@13.4.0
```

## 13.25 IMPORTANT Spack Spec Concept

Every Spack installation contains:

- package version
- compiler information
- dependency tree
- unique installation hash

Example:

`openmpi@5.0.7%gcc@13.4.0`

Meaning:

| Component  | Meaning         |
|------------|-----------------|
| openmpi    | package         |
| 5.0.7      | package version |
| gcc@13.4.0 | compiler used   |

## 13.26 IMPORTANT Hash-Based Software Loading

Sometimes multiple builds of same version exist because of:

- different compilers
- different variants
- different dependencies

Spack assigns:

**unique hashes**

View hashes:

`spack find -l`

Expected:

`abc1234 gcc@13.4.0`

`xyz5678 openmpi@5.0.7`

Load using hash:

`spack load /xyz5678`

This is extremely common in production HPC systems.

### 13.27 Load OpenMPI Environment

```
spack load openmpi
```

Verify:

```
which mpicc
```

Expected:

- Spack OpenMPI wrapper path

Verify MPI runtime:

```
mpirun --version
```

Expected:

- OpenMPI version information

### 13.28 IMPORTANT MPI Wrapper Concept

```
mpicc
```

is NOT a compiler itself.

It is a wrapper that automatically injects:

- MPI headers
- MPI libraries
- runtime linker flags

around GCC.

This is a critical HPC build concept.

### 13.29 Verify Loaded Software

Display loaded packages:

```
spack find --loaded
```

Unload OpenMPI:

```
spack unload openmpi
```

Unload all software:

```
spack unload --all
```

Verify:

```
spack find --loaded
```

Expected:

- no loaded packages

### 13.30 Create MPI Performance Validation Program

Create source file:

```
vim mpi_sum.c
```

Purpose:

- validate MPI runtime
- validate multinode execution
- validate scheduler-aware execution
- compare serial vs parallel runtime
- demonstrate reduction operations
- calculate speedup
- calculate efficiency

Paste:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000000

double serial_sum()
{
```

```

    double sum = 0.0;

    for(long long int i = 0; i < N; i++)
    {
        sum += i;
    }

    return sum;
}

int main(int argc, char *argv[])
{
    int rank, size;

    long long int start_index;
    long long int end_index;
    long long int chunk;

    double local_sum = 0.0;
    double global_sum = 0.0;

    double serial_start;
    double serial_end;

    double parallel_start;
    double parallel_end;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank == 0)
    {
        serial_start = MPI_Wtime();

        double serial_result = serial_sum();

        serial_end = MPI_Wtime();
    }
}

```



```

        printf("\n");
        printf("===== SERIAL COMPUTATION =====\n");
        printf("Sum = %.0f\n", serial_result);
        printf("Time taken = %f seconds\n",
                serial_end - serial_start);
    }

    MPI_Barrier(MPI_COMM_WORLD);

    chunk = N / size;

    start_index = rank * chunk;

    if(rank == size - 1)
        end_index = N;
    else
        end_index = start_index + chunk;

    parallel_start = MPI_Wtime();

    for(long long int i = start_index;
        i < end_index;
        i++)
    {
        local_sum += i;
    }

    MPI_Reduce(&local_sum,
               &global_sum,
               1,
               MPI_DOUBLE,
               MPI_SUM,
               0,
               MPI_COMM_WORLD);

    parallel_end = MPI_Wtime();

    if(rank == 0)
    {
        double parallel_time =

```

```

        parallel_end - parallel_start;

double serial_time =
    serial_end - serial_start;

printf("\n");
printf("===== PARALLEL COMPUTATION =====\n");
printf("Processes used = %d\n", size);
printf("Sum = %.0f\n", global_sum);
printf("Time taken = %f seconds\n",
        parallel_time);

printf("\n");
printf("===== PERFORMANCE =====\n");
printf("Speedup = %f\n",
        serial_time / parallel_time);

printf("Efficiency = %f\n",
        (serial_time / parallel_time) / size);
}

MPI_Finalize();

return 0;
}

```

### 13.31 IMPORTANT MPI Reduction Concept

This program uses:

`MPI_Reduce()`

Purpose:

Combine partial sums from all MPI ranks into:

single global result

Flow:

local sums

↓

`MPI_Reduce`

↓

`global sum`

This is one of the MOST important MPI collective communication operations.

### 13.32 IMPORTANT Parallel Execution Concept

Serial execution:

`single process computes entire workload`

Parallel execution:

`multiple MPI ranks divide workload`

Example:

| Rank  | Work        |
|-------|-------------|
| rank0 | 0 -> 25%    |
| rank1 | 25% -> 50%  |
| rank2 | 50% -> 75%  |
| rank3 | 75% -> 100% |

Advantages:

- reduced execution time
- distributed workload
- scalable computation

### 13.33 Load Software Before Compilation

Initialize Spack:

`source /apps/spack/share/spack/setup-env.sh`

Load compiler:

`spack load gcc@13.4.0`

Load MPI runtime:

```
spack load openmpi
```

Verify:

```
which mpicc
```

Expected:

- MPI compiler wrapper path

### 13.34 Compile MPI Program

Compile:

```
mpicc mpi_sum.c -o mpi_sum
```

Verify executable:

```
ls
```

Expected:

```
mpi_sum
```

### 13.35 IMPORTANT MPI Compilation Concept

MPI programs are compiled using:

```
mpicc
```

instead of:

```
gcc
```

because MPI wrapper automatically injects:

- MPI headers
- MPI libraries
- linker flags

### 13.36 Interactive MPI Runtime Validation

Run interactively:

```
mpirun -np 4 ./mpi_sum
```

Expected:

```
===== SERIAL COMPUTATION =====
```

```
...
```

```
===== PARALLEL COMPUTATION =====
```

```
...
```

```
===== PERFORMANCE =====
```

```
...
```

This validates:

- MPI runtime
- MPI communication
- reduction operations
- distributed execution

### 13.37 IMPORTANT Shared Filesystem Advantage

Because:

- source code
- executable
- software stack

exist on shared filesystem:

```
/apps
```

```
or
```

```
/home
```

compute nodes automatically access:

- compiled binaries
- runtime libraries
- MPI stack

without recompilation.

This is one of the major HPC architecture advantages.

### 13.38 Create Slurm Batch Script

Create batch script:

```
vim submit.sh
```

Paste:

```
#!/bin/bash
#SBATCH -N 3
#SBATCH --ntasks-per-node=1
#SBATCH --job-name=test
#SBATCH --output=%J.out
#SBATCH --error=%J.err
#SBATCH --time=01:00:00
##SBATCH --odelist=compute03,compute01

source /apps/spack/share/spack/setup-env.sh

spack load openmpi

mpirun -np $SLURM_NTASKS ./ $1

Make executable:

chmod +x submit.sh
```

### 13.39 IMPORTANT Slurm Script Explanation

| Line                | Purpose               |
|---------------------|-----------------------|
| #SBATCH -N 3        | request 3 nodes       |
| --ntasks-per-node=1 | one MPI rank per node |
| --job-name=test     | job name              |
| --output=%J.out     | stdout file           |
| --error=%J.err      | stderr file           |
| --time=01:00:00     | walltime limit        |

## 13.40 IMPORTANT Runtime Environment Section

Inside batch script:

```
source /apps/spack/share/spack/setup-env.sh
```

initializes Spack environment.

```
spack load openmpi
```

loads MPI runtime stack.

Without these:

- mpirun may fail
- MPI libraries may not load
- runtime linker errors may occur

## 13.41 IMPORTANT Slurm Runtime Variable

`$SLURM_NTASKS`

contains:

total MPI process count allocated by Slurm

Example:

| Nodes | Tasks per node | Total |
|-------|----------------|-------|
| 3     | 1              | 3     |

Therefore:

```
mpirun -np $SLURM_NTASKS
```

automatically launches correct number of MPI ranks.

### 13.42 Submit MPI Job

Submit job:

```
sbatch submit.sh mpi_sum
```

Expected:

```
Submitted batch job 101
```

This validates:

- Slurm scheduler
- distributed allocation
- multinode MPI execution
- scheduler-aware runtime launch

### 13.43 Monitor Jobs

Check user jobs:

```
squeue --me
```

Expected:

```
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
```

### 13.44 IMPORTANT Job States

| State | Meaning    |
|-------|------------|
| PD    | pending    |
| R     | running    |
| CG    | completing |
| CD    | completed  |
| F     | failed     |



### 13.45 View Detailed Job Information

Show detailed job info:

```
scontrol show job <jobid>
```

Example:

```
scontrol show job 101
```

Displays:

- allocated nodes
- CPUs
- runtime
- scheduling information
- execution directory
- output locations

### 13.46 Check Cluster Status

Check cluster nodes:

```
sinfo
```

Expected:

```
debug* up infinite 3 idle compute[01-03]
```

### 13.47 IMPORTANT Node States

| State | Meaning        |
|-------|----------------|
| idle  | available      |
| alloc | allocated      |
| mix   | partially used |
| drain | unavailable    |
| down  | offline        |

### 13.48 Check Output Files

After completion:

```
ls
```

Expected:

```
101.out
```

```
101.err
```

View output:

```
cat 101.out
```

Expected:

- serial runtime
- parallel runtime
- speedup
- efficiency

### 13.49 IMPORTANT HPC Performance Metrics

The program calculates:

| Metric     | Meaning                                       |
|------------|-----------------------------------------------|
| Speedup    | $\text{serial\_time} / \text{parallel\_time}$ |
| Efficiency | $\text{speedup} / \text{processes}$           |

Ideal values:

| Metric     | Ideal              |
|------------|--------------------|
| Speedup    | near process count |
| Efficiency | near 1.0           |

Real systems experience:

- communication overhead
- synchronization cost
- MPI latency
- reduction overhead

### 13.50 Cancel Jobs

Cancel job:

```
scancel <jobid>
```

Example:

```
scancel 101
```

Cancel all user jobs:

```
scancel -u $USER
```

### 13.51 Interactive Slurm Execution

Launch interactive shell:

```
srun --pty bash
```

Distributed hostname test:

```
srun -N 3 -n 3 hostname
```

Expected:

- hostnames from allocated nodes

This validates:

- scheduler allocation
- distributed execution
- runtime communication

### 13.52 IMPORTANT HPC Workflow

Typical HPC workflow:

```
Load software
  ↓
Compile application
  ↓
Prepare Slurm script
  ↓
Submit job
  ↓
Monitor execution
  ↓
Analyze outputs
```

### 13.53 IMPORTANT Production HPC Concept

Production HPC systems usually separate:

- operating system packages
- scientific software stack
- compiler stack
- MPI runtime stack
- scheduler integration

exactly like this setup.

This architecture provides:

- reproducibility
- scalability
- centralized administration
- software consistency
- runtime portability

## 14 Centralized Identity Management using OpenLDAP + nslcd

Goal:

Transform the cluster from:

`local-user-only infrastructure`

into:

`centralized multi-user HPC environment`

using:

- OpenLDAP
- nslcd
- NSS
- PAM

This phase introduces centralized user management for the HPC cluster.  
After completion:

- users are created once centrally
- login node recognizes LDAP users
- compute nodes recognize LDAP users
- centralized authentication works cluster-wide
- automatic home directory creation works
- Slurm can later use centralized identities

## 14.1 Important Architecture

Cluster roles:

| Node      | Role        |
|-----------|-------------|
| master    | LDAP server |
| login     | LDAP client |
| compute01 | LDAP client |
| compute02 | LDAP client |
| compute03 | LDAP client |

Important concept:

Only master runs LDAP SERVER (slapd).

Login and compute nodes run LDAP CLIENTS (nslcd).

## 14.2 Important Linux Authentication Concepts

Before starting, understand the layers.

| Component | Purpose                       |
|-----------|-------------------------------|
| LDAP      | centralized user database     |
| NSS       | user/group lookup             |
| PAM       | authentication                |
| nslcd     | bridge between Linux and LDAP |

Authentication flow:

```
SSH Login
↓
PAM
↓
nslcd
↓
LDAP server
```

## 14.3 Install OpenLDAP Server on Master Node

Run ONLY on:

master

Install LDAP packages:

```
sudo dnf install -y \  
openldap \  
openldap-servers \  
openldap-clients
```

Package explanation:

| Package          | Purpose                 |
|------------------|-------------------------|
| openldap         | LDAP utilities          |
| openldap-servers | slapd server daemon     |
| openldap-clients | ldapsearch, ldapadd etc |

## 14.4 Enable LDAP Server

Run ONLY on:

**master**

Enable and start slapd:

```
sudo systemctl enable --now slapd
```

Verify:

```
systemctl status slapd
```

Expected:

**active (running)**

## 14.5 Generate LDAP Admin Password Hash

Run ONLY on:

**master**

Generate secure LDAP admin password hash:

```
slappasswd
```

Enter desired password.

Example:

cluster123

Example output:

{SSHA}xxxxxxxxxxxxxxxxxx

Important:

Copy this hash carefully.

It will be used in LDAP database configuration.

## 14.6 Configure LDAP Database

Run ONLY on:

master

Create LDAP database configuration file:

vim ~/db.ldif

Paste:

```
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcSuffix
olcSuffix: dc=hpc,dc=local
```

```
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcRootDN
olcRootDN: cn=Manager,dc=hpc,dc=local
```

```
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcRootPW
olcRootPW: PASTE_HASH_HERE
```



Replace:

PASTE\_HASH\_HERE

with the hash generated using:

slappasswd

## 14.7 Important LDAP Concepts

LDAP uses a hierarchical tree called:

DIT -- Directory Information Tree

Example structure:

```
dc=hpc,dc=local
ou=People
ou=Groups
```

Meaning of terms:

| Term | Meaning             |
|------|---------------------|
| dc   | domain component    |
| ou   | organizational unit |
| cn   | common name         |

## 14.8 Apply LDAP Database Configuration

Run ONLY on:

master

Apply configuration:

```
sudo ldapmodify -Y EXTERNAL -H ldapi:/// \
-f ~/db.ldif
```

Expected:

modifying entry

## 14.9 Import LDAP Schemas

Schemas define:

- user attributes
- group attributes
- UNIX account structure

Run ONLY on:

**master**

Import cosine schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// \  
-f /etc/openldap/schema/cosine.ldif
```

Import nis schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// \  
-f /etc/openldap/schema/nis.ldif
```

Import inetorgperson schema:

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// \  
-f /etc/openldap/schema/inetorgperson.ldif
```

Schema explanation:

| Schema        | Purpose                      |
|---------------|------------------------------|
| cosine        | basic directory attributes   |
| nis           | UNIX/Linux account support   |
| inetorgperson | user/person identity support |

## 14.10 Create LDAP Base Tree

Run ONLY on:

**master**

Create base LDAP tree:

```
vim ~/base.ldif
```

Paste:

```
dn: dc=hpc,dc=local
objectClass: top
objectClass: dcObject
objectClass: organization
o: HPC Cluster
dc: hpc

dn: ou=People,dc=hpc,dc=local
objectClass: organizationalUnit
ou: People

dn: ou=Groups,dc=hpc,dc=local
objectClass: organizationalUnit
ou: Groups
```

Apply tree:

```
ldapadd -x \
-D "cn=Manager,dc=hpc,dc=local" \
-W \
-f ~/base.ldif
```

Enter LDAP admin password.

## 14.11 Verify LDAP Tree

Run ONLY on:

master

Verify tree:

```
ldapsearch -x -b "dc=hpc,dc=local"
```

Expected:

- dc=hpc,dc=local
- ou=People
- ou=Groups

## 14.12 Create LDAP Group

Run ONLY on:

master

Create group file:

```
vim ~/group.ldif
```

Paste:

```
dn: cn=hpcusers,ou=Groups,dc=hpc,dc=local
objectClass: posixGroup
cn: hpcusers
gidNumber: 10000
```

Add group:

```
ldapadd -x \
-D "cn=Manager,dc=hpc,dc=local" \
-W \
-f ~/group.ldif
```

## 14.13 Create LDAP User Password Hash

Run ONLY on:

master

Generate password hash:

```
slappasswd
```

Example password:

```
test123
```

Copy generated hash carefully.

## 14.14 Create LDAP User

Run ONLY on:

master

Create user file:

```
vim ~/user.ldif
```

Paste:

```
dn: uid=testUser,ou=People,dc=hpc,dc=local
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: Test User
sn: User
uid: testUser
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/testUser
loginShell: /bin/bash
userPassword: PASTE_HASH_HERE
```

Replace:

PASTE\_HASH\_HERE

with actual hash from:

```
slappasswd
```

Add user:

```
ldapadd -x \
-D "cn=Manager,dc=hpc,dc=local" \
-W \
-f ~/user.ldif
```

## 14.15 Important LDAP User Attributes

| Attribute     | Purpose       |
|---------------|---------------|
| uid           | username      |
| uidNumber     | UNIX UID      |
| gidNumber     | primary group |
| homeDirectory | user home     |
| loginShell    | login shell   |
| cn            | common name   |
| sn            | surname       |

## 14.16 Verify LDAP User

Run ONLY on:

master

Verify user:

```
ldapsearch -x \  
-b "dc=hpc,dc=local" uid=testUser
```

Expected:

- LDAP user entry

## 14.17 Configure LDAP Client on Login Node

Now begin client-side integration.

Run ONLY on:

login

Install LDAP client packages:

```
sudo dnf install -y \  
openldap-clients \  
nss-pam-ldapd
```

## 14.18 Configure nslcd

Run ONLY on:

login

Create configuration:

```
sudo vim /etc/nslcd.conf
```

Paste:

```
uid nslcd
gid ldap
```

```
uri ldap://master
base dc=hpc,dc=local
```

Explanation:

| Parameter | Purpose          |
|-----------|------------------|
| uri       | LDAP server      |
| base      | LDAP search root |

Secure configuration:

```
sudo chmod 600 /etc/nslcd.conf
```

## 14.19 Enable nslcd

Run ONLY on:

login

Enable service:

```
sudo systemctl enable --now nslcd
```

Verify:

```
systemctl status nslcd
```

Expected:

```
active (running)
```

## 14.20 Verify LDAP Connectivity

Run ONLY on:

login

Test LDAP communication:

```
ldapsearch -x \  
-H ldap://master \  
-b "dc=hpc,dc=local" uid=testUser
```

Expected:

- LDAP user entry

This verifies:

- login node can communicate with master LDAP server

## 14.21 Configure NSS

Run ONLY on:

login

Edit:

```
sudo vim /etc/nsswitch.conf
```

Modify lines:

```
passwd: files ldap  
group: files ldap  
shadow: files ldap
```

Important:

This tells Linux:

check local files first, then LDAP.



## 14.22 Restart nslcd

Run ONLY on:

login

Restart service:

```
sudo systemctl restart nslcd
```

## 14.23 Verify LDAP User Resolution

Run ONLY on:

login

Test centralized user lookup:

```
getent passwd testUser
```

Expected:

```
testUser:x:10001:10000:Test User:/home/testUser:/bin/bash
```

Important concept:

This proves Linux itself recognizes LDAP users.

This is one of the most important LDAP verification steps.

## 14.24 NOTE (Do this for master too)

Now we must configure: **master** with EXACT same setup (master should be able to recognise the user).

## 14.25 Install Automatic Home Directory Support

Run ONLY on:

login

Install package:

```
sudo dnf install -y oddjob-mkhomedir
```

Enable service:

```
sudo systemctl enable --now oddjobd
```

Purpose:

Automatically creates user home directory during first successful login.

## 14.26 Configure PAM Authentication

Run ONLY on:

login

Edit:

```
sudo vim /etc/pam.d/system-auth
```

Add LDAP lines in appropriate sections.

Add in auth section:

```
auth          sufficient    pam_ldap.so use_first_pass
```

Add in account section:

```
account       sufficient    pam_ldap.so
```

Add in password section:

```
password      sufficient    pam_ldap.so use_authtok
```

Add in session section:

```
session       optional      pam_ldap.so
session       optional      pam_mkhomedir.so
```

Important:

Do NOT remove pam\_unix.so lines.

## 14.27 Configure Second PAM File

Run ONLY on:

login

Edit:

```
sudo vim /etc/pam.d/password-auth
```

Add SAME LDAP PAM lines.

Reason:

Different login methods use different PAM stacks.

## 14.28 Restart Services

Run ONLY on:

login

Restart services:

```
sudo systemctl restart nslcd
sudo systemctl restart oddjobd
```

## 14.29 Verify LDAP Authentication

Run ONLY on:

login

Test login:

```
su - testUser
```

Enter LDAP password.

Expected:

Creating directory '/home/testUser'

Successful login indicates:

- LDAP authentication working
- PAM integration working
- automatic home directory creation working

### 14.30 Verify User Home

Run ONLY on:

```
login
```

After login:

```
pwd
```

Expected:

```
/home/testUser
```

#### 1. NOW WE MUST REPLICATE TO COMPUTE NODES

Currently:

```
login node works fully
```

Now we configure:

- compute01
- compute02
- compute03

with EXACT same client setup.

### 14.31 Troubleshooting

#### 1. Important Slurm + LDAP Troubleshooting

This section explains one of the MOST common HPC setup issues.

#### 2. Symptom

LDAP users:

- can SSH login
- can use su
- appear in getent passwd

BUT:

```
sbatch submit.sh
```

results in:

- jobs stuck in pending
- BeginTime state
- scheduler instability
- sinfo hanging
- Slurm credential errors
- slurmctld crashes

### 3. Root Cause

Slurm internally works using:

- UID
- GID
- NSS identity resolution

When launching jobs, Slurm performs internal user identity lookups.

If master node cannot resolve LDAP users correctly through:

- NSS
- nslcd
- LDAP

then:

- Slurm credential generation fails
- scheduler may crash
- jobs cannot launch

Common error:

```
_fill_cred_gids: getpwuid failed for uid=10001
```

Meaning:

Slurm cannot resolve LDAP user identity correctly.

#### 4. Important Verification Commands

Run on:

```
master
login
compute nodes
```

Verify LDAP resolution:

```
getent passwd testUser
id testUser
```

Expected:

```
uid=10001(testUser) gid=10000(hpcusers)
```

These commands MUST work correctly on ALL cluster nodes.

#### 5. Important Master Node Verification

The master node MUST also resolve LDAP users correctly.

Verify:

```
getent passwd testUser
```

If master cannot resolve LDAP users:

- slurmctld may fail
- scheduler may crash
- jobs remain pending

Verify nslcd configuration on master:

```
sudo vim /etc/nslcd.conf
```

Example:

```
uid nslcd
gid ldap

uri ldap://master
base dc=hpc,dc=local
```

Verify NSS configuration:

```
sudo vim /etc/nsswitch.conf
```

Ensure:

```
passwd: files ldap
group: files ldap
shadow: files ldap
```

Restart LDAP client:

```
sudo systemctl restart nslcd
```

## 6. Important Slurm Architecture

Correct architecture:

| Node      | Service             |
|-----------|---------------------|
| master    | slurmctld           |
| login     | login services only |
| compute01 | slurmd              |
| compute02 | slurmd              |
| compute03 | slurmd              |

IMPORTANT:

Login node should NOT run slurmd.

Because login node is NOT configured as a compute node in:

```
/etc/slurm/slurm.conf
```

Attempting to start slurmd on login gives:

```
slurmd: fatal: Unable to determine this slurmd's NodeName
```

This is NORMAL and expected.

Correct action on login node:

```
sudo systemctl stop slurmd
sudo systemctl disable slurmd
```

Expected afterward:

```
inactive (dead)
```

This is CORRECT for login nodes.

## 7. Important Slurm Recovery Commands

Whenever:

- LDAP users are added
- NSS configuration changes
- PAM configuration changes
- nslcd configuration changes

restart Slurm services.

Restart controller on master:

```
sudo systemctl restart slurmctld
```

Restart compute daemons on compute nodes:

```
sudo systemctl restart slurmd
```

Restart LDAP client:

```
sudo systemctl restart nslcd
```



Verify cluster:

```
sinfo
```

Expected:

```
debug* up infinite 3 idle compute[01-03]
```

#### 8. Important Scheduler Debugging

If:

```
sinfo
```

hangs indefinitely:

Possible cause:

```
slurmctld crashed
```

Verify:

```
systemctl status slurmctld
```

or:

```
journalctl -u slurmctld -n 50
```

### 14.32 Current Cluster State After LDAP Setup

Cluster now contains:

| Component                  | Status  |
|----------------------------|---------|
| OpenLDAP server            | working |
| centralized users          | working |
| centralized authentication | working |
| nsld integration           | working |
| NSS integration            | working |
| PAM authentication         | working |
| automatic home creation    | working |

### 14.33 Important Debugging Commands

Verify LDAP tree:

```
ldapsearch -x -b "dc=hpc,dc=local"
```

Verify LDAP user:

```
ldapsearch -x -b "dc=hpc,dc=local" uid=testUser
```

Verify LDAP authentication:

```
ldapwhoami -x \  
-D "uid=testUser,ou=People,dc=hpc,dc=local" \  
-W \  
-H ldap://master
```

Verify Linux user resolution:

```
getent passwd testUser
```

Verify nslcd service:

```
systemctl status nslcd
```

### 14.34 Important Architecture Understanding

Master node:

- provides centralized identity database

Login/compute nodes:

- authenticate users
- resolve LDAP users
- create user sessions

This separation is fundamental in distributed authentication systems.

## 15 Automated LDAP User Creation Script

Managing users manually using individual LDIF files becomes difficult as the number of users increases.

To simplify user management, an automation script can be used to:

- create LDAP group
- create LDAP user
- generate password automatically
- generate password hash automatically
- add entries into LDAP

This script automates complete LDAP user creation.

IMPORTANT:

The script creates:

- private group for user
- LDAP user entry
- hashed LDAP password

automatically.

### 15.1 Script Location

Example:

```
cd ~/ldap-scripts  
ls
```

Expected:

```
createUser.sh
```

## 15.2 User Creation Script

Create file:

```
vim createUser.sh
```

Paste the complete script exactly as shown below:

```
#!/bin/bash

LDAP_ADMIN="cn=Manager,dc=hpc,dc=local"
BASE_DN="dc=hpc,dc=local"

USERNAME=$1
UIDNUM=$2
GIDNUM=$3

if [ $# -ne 3 ]; then
    echo "Usage: $0 <username> <uid> <gid>"
    exit 1
fi

# Check if user already exists
USER_EXISTS=$(ldapsearch -x \
-b "ou=People,$BASE_DN" "(uid=${USERNAME})" dn \
| grep "^dn:")

if [ ! -z "$USER_EXISTS" ]; then
    echo "ERROR: User ${USERNAME} already exists!"
    exit 1
fi

# Check if group already exists
GROUP_EXISTS=$(ldapsearch -x \
-b "ou=Groups,$BASE_DN" "(cn=${USERNAME})" dn \
| grep "^dn:")

if [ ! -z "$GROUP_EXISTS" ]; then
    echo "ERROR: Group ${USERNAME} already exists!"
    exit 1
fi
```

```

PASSWORD="${USERNAME}@0123"

PASSWORD_HASH=$(slappasswd -s "$PASSWORD")

echo
echo "=====
echo "Creating LDAP User"
echo "=====
echo "Username : ${USERNAME}"
echo "UID      : ${UIDNUM}"
echo "GID      : ${GIDNUM}"
echo "=====

# Create private group LDIF
cat > /tmp/${USERNAME}_group.ldif <<EOF
dn: cn=${USERNAME},ou=Groups,${BASE_DN}
objectClass: posixGroup
cn: ${USERNAME}
gidNumber: ${GIDNUM}
EOF

# Create user LDIF
cat > /tmp/${USERNAME}.ldif <<EOF
dn: uid=${USERNAME},ou=People,${BASE_DN}
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: ${USERNAME}
sn: ${USERNAME}
uid: ${USERNAME}
uidNumber: ${UIDNUM}
gidNumber: ${GIDNUM}
homeDirectory: /home/${USERNAME}
loginShell: /bin/bash
userPassword: ${PASSWORD_HASH}
EOF

echo
echo "Adding LDAP group..."

```

```

ldapadd -x \
-D "${LDAP_ADMIN}" \
-W \
-f /tmp/${USERNAME}_group.ldif

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to create LDAP group!"
    exit 1
fi

echo
echo "Adding LDAP user..."

ldapadd -x \
-D "${LDAP_ADMIN}" \
-W \
-f /tmp/${USERNAME}.ldif

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to create LDAP user!"
    exit 1
fi

echo
echo "=====
echo "LDAP User Created Successfully"
echo "=====
echo "Username : ${USERNAME}"
echo "Password : ${PASSWORD}"
echo "UID      : ${UIDNUM}"
echo "GID      : ${GIDNUM}"
echo "=====

```

### 15.3 Make Script Executable

Before using the script:

```
chmod +x createUser.sh
```

## 15.4 Understanding Script Arguments

The script requires THREE arguments:

| Argument | Meaning       |
|----------|---------------|
| username | LDAP username |
| uid      | UNIX UID      |
| gid      | UNIX GID      |

Syntax:

```
./createUser.sh <username> <uid> <gid>
```

Example:

```
./createUser.sh student01 11001 11001
```

Explanation:

| Value     | Meaning  |
|-----------|----------|
| student01 | username |
| 11001     | UID      |
| 11001     | GID      |

IMPORTANT:

The script creates a private group automatically.

Therefore username and group name become identical.

Example:

| Entity         | Created Automatically |
|----------------|-----------------------|
| User           | student01             |
| Group          | student01             |
| Home Directory | /home/student01       |

## 15.5 Default Password Format

The script automatically creates password using:

`username@@123`

Example:

| Username  | Password       |
|-----------|----------------|
| student01 | student01@@123 |
| student02 | student02@@123 |

IMPORTANT:

Users should change password after first login.

## 15.6 What the Script Automatically Does

The script performs following operations automatically:

| Operation              | Description             |
|------------------------|-------------------------|
| Check existing user    | avoids duplicate users  |
| Check existing group   | avoids duplicate groups |
| Generate password hash | secure LDAP password    |
| Create LDAP group LDIF | automatic               |
| Create LDAP user LDIF  | automatic               |
| Add LDAP group         | automatic               |
| Add LDAP user          | automatic               |

## 15.7 Example User Creation

Example:

```
./createUser.sh student02 11002 11002
```

Expected output:

```
=====
Creating LDAP User
=====
Username : student02
UID      : 11002
GID      : 11002
=====
```



LDAP admin password will be requested.

After successful creation:

```
=====
LDAP User Created Successfully
=====
Username : student02
Password : student02@@123
UID      : 11002
GID      : 11002
=====
```

## 15.8 Verify User Creation

Verify user using:

```
getent passwd student02
```

or:

```
id student02
```

Expected:

```
uid=11002(student02)
```

Verify LDAP entry:

```
ldapsearch -x \
-b "dc=hpc,dc=local" uid=student02
```

## 15.9 Verify Login

Test login:

```
su - student02
```

or:

```
ssh student02@login
```

Expected:

```
successful login
```

## 15.10 Important UID/GID Notes

IMPORTANT:

UID and GID values must remain unique cluster-wide.

Avoid:

- duplicate UID
- duplicate GID

because duplicate IDs can create:

- ownership conflicts
- wrong usernames
- incorrect file ownership
- Slurm identity problems

Example of BAD configuration:

| User      | UID   |
|-----------|-------|
| student01 | 11001 |
| student02 | 11001 |

This can cause:

`whoami` returning wrong username

Always use unique UID/GID values.

## 15.11 Important HPC Identity Concept

In HPC clusters:

- NFS
- Slurm
- MPI
- LDAP

- SSH

all depend on:

`consistent unique UID/GID mappings`

across the entire cluster.

This is one of the MOST important concepts in centralized HPC authentication.

## 16 Automated LDAP User Deletion Script

Managing users manually using:

- `ldapdelete`
- group deletion
- home directory cleanup

can become repetitive and error-prone.

To simplify cleanup operations, an automated LDAP deletion script can be used.

This script automates:

- LDAP user deletion
- optional LDAP group deletion
- optional home directory deletion

IMPORTANT:

The script provides interactive cleanup options for safer user management.

### 16.1 Script Location

Example:

```
cd ~/ldap-scripts
ls
```

Expected:

```
deleteUser.sh
```

## 16.2 User Deletion Script

Create file:

```
vim deleteUser.sh
```

Paste the complete script exactly as shown below:

```
#!/bin/bash

LDAP_ADMIN="cn=Manager,dc=hpc,dc=local"
BASE_DN="dc=hpc,dc=local"

USERNAME=$1

if [ $# -ne 1 ]; then
    echo "Usage: $0 <username>"
    exit 1
fi

echo
echo "=====
echo "Deleting LDAP User"
echo "=====
echo "Username : ${USERNAME}"
echo "=====

echo
echo "Deleting LDAP user entry..."

ldapdelete -x \
-D "${LDAP_ADMIN}" \
-W \
"uid=${USERNAME},ou=People,${BASE_DN}"

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to delete LDAP user!"
    exit 1
fi

echo
```

```

echo "LDAP user deleted successfully."

# Ask about deleting group
echo
read -p "Delete LDAP group '${USERNAME}'? (y/n): " DELETE_GROUP

case $DELETE_GROUP in
    y|Y)

        ldapdelete -x \
        -D "${LDAP_ADMIN}" \
        -W \
        "cn=${USERNAME},ou=Groups,${BASE_DN}"

        if [ $? -eq 0 ]; then
            echo "LDAP group deleted successfully."
        else
            echo "WARNING: Failed to delete LDAP group."
        fi
        ;;

    n|N)
        echo "Skipping LDAP group deletion."
        ;;

    *)
        echo "Invalid option. Skipping group deletion."
        ;;
esac

# Ask about deleting home directory
echo
read -p "Delete home directory '/home/${USERNAME}'? (y/n): " DELETE_HOME

case $DELETE_HOME in
    y|Y)

        if [ -d "/home/${USERNAME}" ]; then
            sudo rm -rf "/home/${USERNAME}"

```

```

        if [ $? -eq 0 ]; then
            echo "Home directory deleted successfully."
        else
            echo "WARNING: Failed to delete home directory."
        fi
    else
        echo "Home directory does not exist."
    fi
;;

n|N)
    echo "Skipping home directory deletion."
;;

*)
    echo "Invalid option. Skipping home deletion."
;;
esac

echo
echo "====="
echo "User cleanup completed"
echo "====="

```

### 16.3 Make Script Executable

Before using the script:

```
chmod +x deleteUser.sh
```

### 16.4 Understanding Script Arguments

The script requires ONE argument:

| Argument | Meaning       |
|----------|---------------|
| username | LDAP username |

Syntax:

```
./deleteUser.sh <username>
```

Example:

```
./deleteUser.sh student02
```

## 16.5 What the Script Automatically Does

The script performs following operations automatically:

| Operation                | Description                     |
|--------------------------|---------------------------------|
| Delete LDAP user         | removes LDAP account            |
| Optional group deletion  | removes LDAP group if requested |
| Optional home deletion   | removes user home if requested  |
| Interactive confirmation | prevents accidental cleanup     |

## 16.6 Example User Deletion

Example:

```
./deleteUser.sh student02
```

Expected output:

```
=====
Deleting LDAP User
=====
Username : student02
=====
```

LDAP admin password will be requested.  
After successful deletion:

LDAP user deleted successfully.

The script then asks:

Delete LDAP group 'student02'? (y/n):

If:

y

is selected, corresponding LDAP group will also be removed.  
Next prompt:

Delete home directory '/home/student02'? (y/n):

If:

y

is selected, user home directory will also be deleted.

## 16.7 Important Group Deletion Notes

IMPORTANT:

Do NOT delete shared groups accidentally.

Example:

`hpcusers`

may contain multiple users.

Deleting shared groups can create:

- broken group mappings
- permission problems
- Slurm identity issues

The script therefore asks confirmation before deleting groups.

## 16.8 Important Home Directory Notes

Deleting home directories permanently removes:

- source codes
- job scripts
- outputs
- datasets
- SSH keys
- configuration files

IMPORTANT:

Home directory deletion is irreversible.

Always verify before deleting.



## 16.9 Verify User Removal

Verify deletion:

```
getent passwd student02
```

Expected:

```
no output
```

Verify LDAP entry removal:

```
ldapsearch -x \  
-b "dc=hpc,dc=local" uid=student02
```

Expected:

```
no entries returned
```

Verify login failure:

```
su - student02
```

Expected:

```
user does not exist
```

## 16.10 Important HPC User Management Concept

In centralized HPC environments:

- user lifecycle management
- account cleanup
- UID/GID consistency
- group management

are critical administrative tasks.

Improper deletion can create:

- orphaned files
- broken permissions
- Slurm scheduling issues
- incorrect ownership mappings

Therefore automated and controlled cleanup procedures are highly important in production HPC systems.

## 17 Automated LDAP Password Change Script

Managing passwords manually using LDIF files becomes inconvenient in centralized LDAP environments.

To simplify password management, an automated password change script can be used.

This script automates:

- LDAP password update
- password hash generation
- secure password input
- LDAP password modification

IMPORTANT:

Because authentication is centralized through LDAP, password changes automatically work cluster-wide.

After changing password:

- login node recognizes new password
- compute nodes recognize new password
- master recognizes new password

without changing anything individually on nodes.

### 17.1 Script Location

Example:

```
cd ~/ldap-scripts  
ls
```

Expected:

```
changePassword.sh
```

## 17.2 Password Change Script

Create file:

```
vim changePassword.sh
```

Paste the complete script exactly as shown below:

```
#!/bin/bash

LDAP_ADMIN="cn=Manager,dc=hpc,dc=local"
BASE_DN="dc=hpc,dc=local"

USERNAME=$1

if [ $# -ne 1 ]; then
    echo "Usage: $0 <username>"
    exit 1
fi

# Check if user exists
USER_EXISTS=$(ldapsearch -x \
-b "ou=People,$${BASE_DN}" "(uid=${USERNAME})" dn \
| grep "^dn:")

if [ -z "$USER_EXISTS" ]; then
    echo "ERROR: User ${USERNAME} does not exist!"
    exit 1
fi

echo
echo "=====
echo "LDAP Password Change"
echo "=====
echo "Username : ${USERNAME}"
echo "=====

# Take password securely
echo
read -s -p "Enter new password: " PASSWORD1
echo
```

```

read -s -p "Confirm new password: " PASSWORD2
echo

# Check password match
if [ "$PASSWORD1" != "$PASSWORD2" ]; then
    echo
    echo "ERROR: Passwords do not match!"
    exit 1
fi

# Generate LDAP password hash
PASSWORD_HASH=$(slappasswd -s "$PASSWORD1")

# Create LDIF file
cat > /tmp/${USERNAME}_password.ldif <<EOF
dn: uid=${USERNAME},ou=People,${BASE_DN}
changetype: modify
replace: userPassword
userPassword: ${PASSWORD_HASH}
EOF

echo
echo "Updating LDAP password..."

ldapmodify -x \
-D "${LDAP_ADMIN}" \
-W \
-f /tmp/${USERNAME}_password.ldif

if [ $? -ne 0 ]; then
    echo
    echo "ERROR: Failed to change password!"
    exit 1
fi

echo
echo "=====
echo "Password updated successfully"
echo "=====

```

```
echo "Username : ${USERNAME}"
echo "=====
```

### 17.3 Make Script Executable

Before using the script:

```
chmod +x changePassword.sh
```

### 17.4 Understanding Script Arguments

The script requires ONE argument:

| Argument | Meaning       |
|----------|---------------|
| username | LDAP username |

Syntax:

```
./changePassword.sh <username>
```

Example:

```
./changePassword.sh student01
```

### 17.5 What the Script Automatically Does

The script performs following operations automatically:

| Operation               | Description                    |
|-------------------------|--------------------------------|
| Verify LDAP user exists | prevents invalid modifications |
| Secure password input   | password hidden during typing  |
| Password confirmation   | avoids typing mistakes         |
| Generate password hash  | secure LDAP password storage   |
| Modify LDAP password    | centralized password update    |

### 17.6 Example Password Change

Example:

```
./changePassword.sh student01
```

Expected output:

```
=====
LDAP Password Change
=====
Username : student01
=====
```

The script asks:

Enter new password:

Then:

Confirm new password:

LDAP admin password will also be requested.

After successful update:

```
=====
Password updated successfully
=====
Username : student01
=====
```

## 17.7 Verify Password Change

Test login using updated password:

```
su - student01
```

or:

```
ssh student01@login
```

Expected:

```
successful login using new password
```

Because LDAP authentication is centralized, the updated password works automatically on:

- login node
- compute nodes
- master node

## 17.8 Important Password Management Notes

IMPORTANT:

LDAP stores password hashes, not plain-text passwords.

The script automatically generates secure LDAP-compatible password hashes using:

```
slappasswd
```

This improves security by avoiding direct plain-text password storage.

## 18 Advanced Slurm Configuration Preparation

This phase marks the transition from a:

```
basic educational Slurm setup
```

to a more:

```
resource-aware and production-like HPC scheduler environment
```

At this stage, the cluster was already fully functional and stable.  
Existing working components:

- Slurm scheduler
- sbatch job submission
- MPI execution
- multinode MPI jobs
- LDAP centralized authentication
- shared filesystem
- shared Spack software stack

The goal of this phase was to carefully introduce advanced scheduler features WITHOUT destabilizing runtime execution.

## 18.1 IMPORTANT APPROACH

Advanced Slurm administration should always be performed incrementally.

Instead of enabling many advanced features simultaneously, only ONE feature was introduced and validated at a time.

This approach helps:

- isolate issues properly
- simplify debugging
- understand scheduler behavior
- maintain cluster stability

This is how production HPC administrators commonly manage scheduler configuration changes.

## 18.2 Advanced Slurm Roadmap

The cluster was upgraded gradually in the following order:

| Phase | Feature                      | Risk Level |
|-------|------------------------------|------------|
| 1     | Resource Accounting          | Safe       |
| 2     | Memory-Aware Scheduling      | Safe       |
| 3     | CPU Binding / Affinity       | Safe       |
| 4     | Job Limits                   | Safe       |
| 5     | QoS                          | Medium     |
| 6     | Accounts & Fairshare         | Medium     |
| 7     | Reservations                 | Medium     |
| 8     | Advanced Scheduling Policies | Medium     |
| 9     | cgroups/task/cgroup          | Advanced   |
| 10    | Monitoring & Metrics         | Advanced   |

### 1. IMPORTANT NOTE

At this stage:

`TaskPlugin=task/cgroup`

was intentionally NOT enabled.

Reason:

Strict cgroup enforcement can introduce runtime instability, especially on:



- Rocky Linux 9
- cgroup v2 systems
- educational clusters

Instead, the cluster first focused on:

- stable scheduling
- resource accounting
- affinity scheduling
- scheduler-aware resource tracking

before moving toward strict kernel-level enforcement.

### 18.3 Resource-Aware Scheduling

Goal:

Teach Slurm to understand:

- CPU resources
- memory resources
- consumable resources
- schedulable resources

without enabling strict runtime enforcement.

This creates a much safer and more stable intermediate scheduler state.

### 18.4 Understanding Current Cluster Resource State

Initial scheduler behavior:

- node-aware scheduling
- CPU-aware scheduling

BUT:

Slurm did NOT properly understand actual node memory capacity.

This can be verified using:

```
scontrol show node compute01
```

Without:

RealMemory

configured, Slurm cannot properly perform memory-aware scheduling.

## 18.5 Understanding Actual Node Memory

Actual node memory was checked using:

```
ssh compute01 free -m
ssh compute02 free -m
ssh compute03 free -m
```

Observed memory:

~1700 MB

IMPORTANT:

Production HPC clusters NEVER allocate 100% physical memory to jobs.

Reason:

The operating system itself requires memory for:

- kernel
- SSH sessions
- Slurm daemons
- filesystem cache
- system services
- background tasks

A portion of memory must always remain reserved for stable node operation.

## 18.6 Configuring RealMemory

Node configuration was updated inside:

```
sudo vim /etc/slurm/slurm.conf
```

Original configuration:

```
NodeName=compute01 CPUs=2 State=UNKNOWN
NodeName=compute02 CPUs=2 State=UNKNOWN
NodeName=compute03 CPUs=2 State=UNKNOWN
```

Updated configuration:

```
NodeName=compute01 CPUs=2 RealMemory=1200 State=UNKNOWN
NodeName=compute02 CPUs=2 RealMemory=1200 State=UNKNOWN
NodeName=compute03 CPUs=2 RealMemory=1200 State=UNKNOWN
```

## 18.7 RealMemory Concept

RealMemory=1200

does NOT represent total physical memory.  
Instead, it defines:

schedulable memory available to jobs

Example:

| Actual Physical Memory | Configured Schedulable Memory |
|------------------------|-------------------------------|
| ~1700 MB               | 1200 MB                       |

This reserved memory helps maintain:

- system stability
- scheduler reliability
- daemon responsiveness
- SSH availability

during heavy workloads.

## 18.8 Resource-Aware Scheduling Concept

After configuring:

`RealMemory`

Slurm now understands:

- CPU resources
- schedulable memory
- consumable resources

instead of only allocating entire nodes.

This is a major scheduler upgrade.

The scheduler transitioned from:

`simple node allocation`

to:

`resource-aware scheduling`

## 18.9 Scheduler Configuration Enhancements

The following important scheduler options were enabled:

`SchedulerType=sched/backfill`

`SelectType=select/cons_tres`

`SelectTypeParameters=CR_Core`

`TaskPlugin=task/affinity`

## 18.10 Configuration Explanation

1. `SchedulerType=sched/backfill`

Enables Slurm backfill scheduler.

Purpose:

Allows smaller jobs to execute while larger jobs wait for resources.

Benefits:

- improves cluster utilization
- reduces idle resources
- increases scheduling efficiency

This is commonly used in production HPC systems.

## 2. SelectType=select/cons\_tres

Enables:

`consumable resource scheduling`

This allows Slurm to track:

- CPUs
- memory
- schedulable resources

instead of only allocating complete nodes.

## 3. SelectTypeParameters=CR\_Core

Enables:

`core-aware scheduling`

Resources are now allocated at CPU-core granularity.

This is extremely important for:

- MPI workloads
- OpenMP workloads
- hybrid parallel applications

## 4. TaskPlugin=task/affinity

Enables CPU affinity support.

Purpose:

Allows Slurm to:

- pin tasks to CPUs

- improve cache locality
- optimize task placement
- stabilize parallel performance

This is one of the most important performance-related scheduler features in HPC environments.

## 18.11 Distributing Updated Slurm Configuration

Updated configuration was distributed cluster-wide.

Copy configuration:

```
scp /etc/slurm/slurm.conf login:/tmp/
scp /etc/slurm/slurm.conf compute01:/tmp/
scp /etc/slurm/slurm.conf compute02:/tmp/
scp /etc/slurm/slurm.conf compute03:/tmp/
```

Move configuration into place:

```
sudo mv /tmp/slurm.conf /etc/slurm/
```

Compute nodes:

```
ssh compute01 sudo -S mv /tmp/slurm.conf /etc/slurm/
ssh compute02 sudo -S mv /tmp/slurm.conf /etc/slurm/
ssh compute03 sudo -S mv /tmp/slurm.conf /etc/slurm/
```

## 18.12 Restarting Slurm Services

Restart controller on master node:

```
sudo systemctl restart slurmctld
```

Restart compute daemons:

```
ssh compute01 sudo -S systemctl restart slurmd
ssh compute02 sudo -S systemctl restart slurmd
ssh compute03 sudo -S systemctl restart slurmd
```

Purpose:

Reload updated scheduler configuration across all nodes.

### 18.13 Verifying Cluster Health

Verify node state:

```
sinfo
```

Expected:

```
compute[01-03] idle
```

Verify scheduler memory recognition:

```
scontrol show node compute01
```

Expected:

```
RealMemory=1200
```

This validates:

- scheduler configuration loaded correctly
- memory-aware scheduling active
- stable cluster operation

### 18.14 Difference Between Scheduling and Enforcement

At this stage:

```
memory-aware scheduling
```

was enabled.

BUT:

```
strict kernel-level memory enforcement
```

was NOT enabled.

This distinction is extremely important.

Current behavior:

```
#SBATCH --mem=500M
```

does:

- reserve memory in scheduler accounting
- affect scheduling decisions
- prevent scheduler over-allocation

BUT does NOT:

- hard-limit memory usage
- kill memory-overusing jobs
- enforce kernel-level restrictions

because:

`TaskPlugin=task/cgroup`

was intentionally NOT enabled.

## 18.15 Why task/cgroup Was Postponed

Earlier experiments with:

`TaskPlugin=task/cgroup`

introduced runtime instability.

Observed issues included:

- batch runtime failures
- node drain states
- unstable job execution
- scheduler inconsistencies

Reason:

Modern Rocky Linux 9 systems use:

`cgroup v2`

which often requires significantly more careful Slurm configuration and kernel-level tuning.

Instead of introducing unstable runtime enforcement early, the cluster was intentionally kept in a stable intermediate state using:



`TaskPlugin=task/affinity`

This approach provides:

- stable runtime execution
- CPU affinity support
- resource-aware scheduling
- simplified administration

without destabilizing scheduler behavior.

## 18.16 Process Tracking Note

The cluster still uses:

`ProctrackType=proctrack/cgroup`

This is generally safe because:

- lightweight process tracking only
- no strict runtime enforcement
- no aggressive kernel-level isolation

This differs significantly from:

`TaskPlugin=task/cgroup`

which performs actual resource enforcement and isolation.

Keeping:

`proctrack/cgroup`

while avoiding:

`task/cgroup`

provides a much safer and more stable educational HPC environment.

## 18.17 Verifying Stable Job Execution

After enabling resource-aware scheduling:

- sbatch jobs remained stable
- MPI jobs executed correctly
- multinode execution continued working
- no node drain issues occurred
- no runtime instability observed

This confirmed that the scheduler upgrade was successful.

## 18.18 Milestone Achieved

The cluster successfully evolved from:

`basic educational scheduler`

into:

`resource-aware production-like scheduler`

while maintaining:

- stable runtime execution
- multinode MPI support
- scheduler reliability
- simplified administration

This forms the foundation required for future advanced features such as:

- CPU affinity and task binding
- QoS
- fairshare
- accounting
- scheduling policies
- advanced resource management

## 19 Advanced Slurm Configuration

This section extends the Slurm setup from a basic scheduler into a more resource-aware and production-like HPC workload manager.

At this stage, the cluster already had:

- working Slurm scheduler
- multinode job execution
- MPI runtime support
- centralized LDAP authentication
- shared filesystem
- shared Spack software stack

The goal of this phase was to gradually introduce advanced scheduler features without destabilizing the cluster runtime environment.

The approach followed:

- incremental configuration
- verification after every change
- understanding scheduler behavior
- avoiding unsafe configurations initially

### 19.1 IMPORTANT APPROACH

Advanced Slurm administration should always be performed incrementally.

Instead of enabling multiple advanced features simultaneously, only one feature was introduced and tested at a time.

This helps isolate issues and understand scheduler behavior properly.

### 19.2 Current Cluster State Before Advanced Configuration

Cluster components already working:

| Component            | Status  |
|----------------------|---------|
| Slurm scheduler      | Working |
| sbatch               | Working |
| MPI jobs             | Working |
| Multinode jobs       | Working |
| LDAP users           | Working |
| Shared filesystem    | Working |
| Spack software stack | Working |

This provided a stable foundation before introducing advanced scheduler features.

### 19.3 Resource-Aware Scheduling

Goal:

Enable Slurm to understand and schedule cluster resources more accurately.

This phase introduced:

- memory-aware scheduling
- consumable resource scheduling
- core-aware scheduling
- scheduler backfill support

without enabling strict kernel-level resource enforcement.

#### 1. IMPORTANT CONCEPT

At this stage:

`memory-aware scheduling`

was enabled.

NOT:

`strict memory enforcement`

This distinction is extremely important.

The scheduler now understands memory resources and schedules jobs accordingly, but Linux is still responsible for actual memory allocation.

No cgroup-based hard memory limits were enabled yet.

## 2. Existing Stable Slurm Configuration

Current working configuration:

```
ClusterName=hpcccluster
SlurmctldHost=master
```

```
MpiDefault=none
ProctrackType=proctrack/cgroup
ReturnToService=2
```

```
SlurmctldPidFile=/run/slurmctld.pid
SlurmdPidFile=/run/slurmd.pid
```

```
SlurmdSpoolDir=/var/spool/slurmd
StateSaveLocation=/var/spool/slurmctld
```

```
SlurmUser=slurm
```

```
SwitchType=switch/none
TaskPlugin=task/affinity
```

```
SchedulerType=sched/backfill
```

```
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
```

```
AuthType=auth/munge
```

```
NodeName=compute01 CPUs=2 RealMemory=1200 State=UNKNOWN
NodeName=compute02 CPUs=2 RealMemory=1200 State=UNKNOWN
NodeName=compute03 CPUs=2 RealMemory=1200 State=UNKNOWN
```

```
PartitionName=debug Nodes=ALL Default=YES MaxTime=INFINITE State=UP
```

## 3. Explanation of Important Parameters

(a) SchedulerType=sched/backfill

Enables Slurm backfill scheduler.

Purpose:

Allows smaller jobs to run while larger jobs wait for resources.

Benefits:

- improves cluster utilization
- reduces idle time
- increases scheduling efficiency

Production HPC systems commonly use backfill scheduling.

(b) SelectType=select/cons\_tres

Enables:

`consumable resource scheduling`

This changes Slurm from:

`simple node allocation`

to:

`resource-aware allocation`

Slurm now tracks:

- CPUs
- memory
- consumable resources

instead of only allocating entire nodes.

(c) SelectTypeParameters=CR\_Core

Enables:

`core-aware scheduling`

Resources are now scheduled at CPU core granularity.

This is extremely important for:

- MPI workloads
- OpenMP workloads
- hybrid parallel jobs

(d) TaskPlugin=task/affinity

Enables CPU affinity support.

Purpose:

Allows Slurm to:

- pin tasks to CPUs
- control process placement
- improve cache locality
- optimize parallel performance

This is one of the most important performance-related scheduler features.

(e) RealMemory=1200

Defines schedulable memory for each node.

Important:

This value should NOT equal total physical memory.

Example:

Node actual memory:

1700 MB

Configured schedulable memory:

1200 MB

Reason:

The operating system itself requires memory for:

- kernel
- filesystem cache
- SSH sessions
- Slurm daemons
- background services

Production clusters always reserve memory for system operation.

#### 4. IMPORTANT MEMORY SCHEDULING CONCEPT

At this stage:

```
#SBATCH --mem=500M
```

does:

- reserve memory in scheduler accounting
- affect scheduling decisions
- prevent scheduler over-allocation

BUT does NOT:

- hard-limit memory usage
- kill memory-overusing jobs
- enforce kernel-level restrictions

because:

`TaskPlugin=task/cgroup`

was intentionally NOT enabled yet.

This avoids runtime instability while still enabling resource-aware scheduling.

## 19.4 CPU Affinity and Task Binding

Goal:

Understand and validate:

- CPU affinity
- process placement
- task binding
- scheduler-controlled CPU allocation

This phase connects directly with:

- parallel performance
- cache locality
- MPI optimization
- OpenMP optimization



## 1. IMPORTANT CONCEPT

Without CPU affinity:

- processes may migrate between CPUs
- cache locality decreases
- performance becomes inconsistent

With affinity:

- processes stay pinned to CPUs
- cache locality improves
- scheduler behavior becomes deterministic

This is extremely important in production HPC systems.

## 19.5 Understanding CPU Topology

CPU topology information was inspected using:

```
ssh compute01 lscpu
```

Important fields:

| Field              | Meaning                 |
|--------------------|-------------------------|
| CPU(s)             | total logical CPUs      |
| Core(s) per socket | physical cores          |
| Socket(s)          | CPU packages            |
| Thread(s) per core | SMT/Hyperthreading      |
| NUMA node(s)       | memory locality domains |

This information becomes important for:

- task placement
- thread pinning
- NUMA optimization
- cache locality

## 19.6 CPU Affinity Validation Program

Program used to validate CPU placement:

```
#define _GNU_SOURCE

#include <stdio.h>
#include <sched.h>
#include <unistd.h>

int main() {

    int cpu = sched_getcpu();

    printf("Process running on CPU: %d\n", cpu);

    sleep(20);

    return 0;
}
```

Compile:

```
gcc affinity.c -o affinity
```

Purpose:

The program asks Linux:

Which CPU core am I running on?

This helps visualize scheduler-controlled CPU placement.

## 19.7 Distributed CPU Placement Test

Slurm batch script:

```
#!/bin/bash
#SBATCH -N 2
#SBATCH --ntasks-per-node=1
#SBATCH --job-name=affinity
#SBATCH --output=%j.out
#SBATCH --error=%j.err
```

```
#SBATCH --time=00:01:00
```

```
srun ./affinity
```

Submit:

```
sbatch affinity.sh
```

Observed behavior:

```
Process running on CPU: 0
```

```
Process running on CPU: 0
```

IMPORTANT:

CPU numbering is local to each node.

Meaning:

- compute01 has CPU0
- compute02 also has CPU0

Each node maintains independent CPU numbering.

This is an important distributed-system concept.

## 19.8 Multiple Task Placement Test

Modified batch script:

```
#!/bin/bash
#SBATCH -N 2
#SBATCH --ntasks-per-node=2
#SBATCH --job-name=affinity
#SBATCH --output=%j.out
#SBATCH --error=%j.err
#SBATCH --time=00:01:00
```

```
srun ./affinity
```

Observed behavior:

```
Process running on CPU: 0
```

```
Process running on CPU: 1
```

```
Process running on CPU: 1
```

```
Process running on CPU: 0
```

This validates:

- task distribution across cores
- scheduler-controlled placement
- multi-core resource allocation

Each node distributed tasks across available CPUs.

## 19.9 Explicit CPU Binding

Explicit CPU binding script:

```
#!/bin/bash
#SBATCH -N 1
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=1
#SBATCH --job-name=bindtest
#SBATCH --output=%j.out
#SBATCH --error=%j.err
#SBATCH --time=00:01:00

srun --cpu-bind=cores ./affinity
```

Submit:

```
sbatch bind.sh
```

Observed behavior:

```
Process running on CPU: 0
Process running on CPU: 1
```

This validates:

- explicit CPU pinning
- deterministic CPU placement
- Slurm affinity control
- Linux scheduler cooperation

## 1. IMPORTANT CONCEPT

Without:

```
--cpu-bind=cores
```

Linux may migrate processes between CPUs.

With explicit binding:

- processes stay pinned
- cache locality improves
- performance stabilizes

This becomes extremely important for:

- MPI scaling
- OpenMP scaling
- hybrid MPI/OpenMP execution
- NUMA systems

## 19.10 Slurm Accounting Database Setup

Goal:

Transform Slurm from:

`basic scheduler`

into:

`persistent accounting-aware scheduler`

This phase introduced:

- MariaDB backend
- Slurm accounting daemon
- persistent job history
- resource usage tracking
- scheduler accounting infrastructure

## 19.11 Install MariaDB

Performed ONLY on master node.

Install packages:

```
sudo dnf install -y mariadb-server mariadb
```

Enable and start service:

```
sudo systemctl enable --now mariadb
```

Verify:

```
sudo systemctl status mariadb
```

Expected:

```
active (running)
```

## 19.12 Secure MariaDB

Run:

```
sudo mysql_secure_installation
```

Recommended configuration:

| Question                             | Recommended Answer |
|--------------------------------------|--------------------|
| Switch to unix_socket authentication | no                 |
| Change root password                 | optional           |
| Remove anonymous users               | yes                |
| Disallow remote root login           | yes                |
| Remove test database                 | yes                |
| Reload privileges                    | yes                |

## 19.13 Create Slurm Accounting Database

Enter MariaDB:

```
sudo mysql -u root -p
```

Create database:

```
CREATE DATABASE slurm_acct_db;
```

Create Slurm database user:

```
CREATE USER 'slurm'@'localhost' IDENTIFIED BY 'slurmdbpass';
```

Grant permissions:

```
GRANT ALL PRIVILEGES ON slurm_acct_db.* TO 'slurm'@'localhost';
```

Apply changes:

```
FLUSH PRIVILEGES;
```

Exit:

```
EXIT;
```

## 19.14 Install Slurm Accounting Daemon

Install:

```
sudo dnf install -y slurm-slurmdbd
```

Purpose:

Installs:

slurmdbd

which connects:

- Slurm scheduler
- accounting database
- resource tracking infrastructure

## 19.15 Configure slurmdbd

Create configuration:

```
sudo vim /etc/slurm/slurmdbd.conf
```

Configuration:

```
AuthType=auth/munge
```

```
DbdHost=master
```

```
DbdPort=6819
```

```
SlurmUser=slurm
```

```
DebugLevel=info
```

```
LogFile=/var/log/slurmdbd.log
```

```
PidFile=/run/slurmdbd.pid
```

```
StorageType=accounting_storage/mysql
```

```
StorageHost=localhost
```

```
StoragePort=3306
```

```
StorageUser=slurm
```

```
StoragePass=slurmdbpass
```

```
StorageLoc=slurm_acct_db
```

### 1. IMPORTANT SECURITY STEP

Protect configuration file:

```
sudo chown slurm:slurm /etc/slurm/slurmdbd.conf
```

```
sudo chmod 600 /etc/slurm/slurmdbd.conf
```

Reason:

Database credentials are stored inside the file.

Production systems always restrict access to this configuration.

## 19.16 Connect Slurm With Accounting System

Modify:



```
sudo vim /etc/slurm/slurm.conf
```

Add:

```
AccountingStorageType=accounting_storage/slurmdbd
AccountingStorageHost=master
AccountingStoragePort=6819
JobAcctGatherType=jobacct_gather/linux
```

Purpose:

Enable:

- persistent job accounting
- resource tracking
- historical scheduler data

## 19.17 Restart Slurm Services

Restart accounting daemon on master:

```
sudo systemctl enable --now slurmdbd
```

Verify:

```
sudo systemctl status slurmdbd
```

Expected:

```
active (running)
```

Restart Slurm controller:

```
sudo systemctl restart slurmctld
```

Restart compute daemons:

```
ssh compute01 sudo -S systemctl restart slurmd
ssh compute02 sudo -S systemctl restart slurmd
ssh compute03 sudo -S systemctl restart slurmd
```

## 19.18 Verify Accounting Infrastructure

Check cluster:

```
sinfo
```

Expected:

```
compute[01-03] idle
```

Check accounting cluster registration:

```
sacctmgr list cluster
```

Expected:

```
hpccluster
```

## 19.19 Verify Persistent Job Accounting

Submit MPI job:

```
sbatch submit.sh a.out
```

Check scheduler queue:

```
squeue --me
```

Check accounting history:

```
sacct
```

Observed output:

| JobID    | JobName | Partition | AllocCPUS | State     |
|----------|---------|-----------|-----------|-----------|
| 22       | test    | debug     | 4         | COMPLETED |
| 22.batch | batch   |           | 2         | COMPLETED |
| 22.0     | prted   |           | 2         | COMPLETED |

This validates:

- persistent accounting
- resource tracking
- MPI runtime tracking
- scheduler database integration

## 19.20 IMPORTANT MPI Accounting Observation

Accounting now tracks internal execution hierarchy:

| Entry    | Meaning              |
|----------|----------------------|
| 22       | main job             |
| 22.batch | batch execution step |
| 22.0     | MPI runtime launcher |

This provides deeper visibility into scheduler behavior and runtime execution structure.

## 19.21 Current Cluster Capability

The cluster now supports:

| Feature                   | Status  |
|---------------------------|---------|
| Multinode scheduling      | Working |
| MPI execution             | Working |
| LDAP authentication       | Working |
| Shared filesystem         | Working |
| Spack software stack      | Working |
| Resource-aware scheduling | Working |
| Memory-aware scheduling   | Working |
| CPU affinity              | Working |
| Explicit CPU binding      | Working |
| Slurm accounting database | Working |
| Persistent job history    | Working |
| Resource tracking         | Working |

The cluster has now evolved from a basic educational setup into a much more realistic HPC infrastructure environment.

## 20 QoS + User Limits + Scheduling Policies

This phase extends the Slurm scheduler from:

`resource-aware scheduling`

into:

`policy-aware scheduling`

At this stage, the cluster already supported:

- multinode scheduling
- MPI execution
- LDAP authentication
- shared filesystem
- shared Spack software stack
- resource-aware scheduling
- CPU affinity
- persistent Slurm accounting

The goal of this phase was to introduce:

- user policies
- scheduler limits
- QoS enforcement
- runtime restrictions
- fair resource sharing

without introducing kernel-level runtime instability.

## 20.1 IMPORTANT CONCEPT

Before this phase:

Slurm could:

- schedule resources
- track jobs
- track users
- record accounting information

BUT it could NOT yet:

- enforce user policies
- restrict CPU usage
- limit runtime
- prevent resource abuse

This phase enables:

`policy-aware scheduling`

similar to real production HPC systems.

## 20.2 IMPORTANT Slurm Scheduling Hierarchy

Slurm scheduling hierarchy:

```
Cluster
  Account
    User
      QoS
```

Production HPC systems commonly organize users using:

- projects
- research groups
- departments
- labs

called:

**Accounts**

Users are then associated with:

- accounts
- QoS policies
- scheduler limits

### 20.3 Existing Stable Accounting Infrastructure

Before enabling QoS enforcement, the following components were already working:

| Component             | Status  |
|-----------------------|---------|
| MariaDB               | Working |
| slurmdbd              | Working |
| sacct                 | Working |
| Persistent accounting | Working |
| Job history tracking  | Working |
| Resource tracking     | Working |

This provided the required foundation for policy enforcement.

### 20.4 Understanding QoS

QoS stands for:

Quality of Service

QoS controls:

- maximum CPUs
- maximum jobs
- walltime limits
- scheduler priorities
- resource restrictions
- policy enforcement

Production HPC clusters heavily rely on QoS to maintain fairness and prevent abuse.

### 20.5 IMPORTANT QoS Concept

QoS does NOT directly enforce kernel-level runtime isolation.

Instead, it performs:

scheduler-side policy enforcement

Meaning:

Slurm scheduler decides:

- whether jobs may start
- whether jobs remain pending
- whether limits are exceeded

WITHOUT requiring:

`task/cgroup`

or strict kernel enforcement.

This keeps the cluster significantly more stable while still enabling realistic scheduler policies.

## 20.6 Verify Cluster Registration

Before creating accounts and policies, verify cluster registration.

Run on:

- login node
- or master node

`sacctmgr list cluster`

Expected:

`hpccluster`

Purpose:

Verifies:

- accounting database connectivity
- slurmdbd functionality
- cluster accounting registration

## 20.7 Create Slurm Account

Accounts represent:

- research groups
- projects
- departments
- user groups

Create a general account.

Run on master node:

```
sudo sacctmgr add account general
```

Verify:

```
sacctmgr list account
```

Expected:

```
general
```

## 20.8 IMPORTANT Account Concept

Linux users and Slurm accounts are NOT the same thing.

Linux users provide:

```
authentication identity
```

Slurm accounts provide:

```
scheduler policy grouping
```

This distinction is extremely important in production HPC systems.



## 20.9 Add Users to Slurm Accounting

LDAP users already existed in Linux.

However, Slurm accounting ALSO requires user associations.

Example user:

`rishi`

Run on master node:

```
sudo sacctmgr add user rishi account=general
```

Add additional users if needed:

```
sudo sacctmgr add user hpcadmin account=general
```

Verify associations:

```
sacctmgr list user
```

Expected:

Users associated with:

`general`

## 20.10 Create QoS Policy

Create QoS policy named:

`normal`

Run on master node:

```
sudo sacctmgr add qos normal
```

Verify:

```
sacctmgr list qos
```

Expected:

`normal`

## 20.11 Configure QoS Limits

The following limits were configured.

### 1. Maximum Concurrent CPUs

Run:

```
sudo sacctmgr modify qos normal set MaxTRESPerUser=cpu=4
```

Purpose:

Limit each user to:

```
maximum 4 CPUs simultaneously
```

IMPORTANT:

This controls:

```
active running resource usage
```

NOT:

- total submissions
- queued jobs
- single-job submission ability

### 2. Maximum Concurrent Running Jobs

Run:

```
sudo sacctmgr modify qos normal set MaxJobsPerUser=2
```

Purpose:

Limit users to:

```
maximum 2 running jobs
```

Additional jobs remain pending.

### 3. Maximum Walltime

Run:

```
sudo sacctmgr modify qos normal set MaxWall=01:00:00
```

Purpose:

Restrict maximum job runtime to:

1 hour

Jobs requesting larger runtime remain pending.

## 20.12 Attach QoS to Account

Connect QoS policy to account.

Run:

```
sudo sacctmgr modify account general set qos=normal
```

Purpose:

Users associated with:

**general**

now inherit:

**normal**

QoS policies.

## 20.13 Verify Associations

Verify:

- user associations
- account associations
- QoS assignments

Run:

`sacctmgr show assoc`

Purpose:

Displays:

- users
- accounts
- partitions
- qos assignments

## 20.14 IMPORTANT ISSUE FACED

Initially, QoS appeared to exist correctly:

- QoS visible
- associations visible
- accounting working
- job tracking working

BUT:

`policy enforcement was NOT working`

Jobs exceeding configured CPU limits still executed successfully.

Example:

`MaxTRESPerUser=cpu=4`

BUT:

`6 CPU jobs still executed`

This revealed an extremely important missing scheduler configuration.

## 20.15 Root Cause of QoS Enforcement Failure

The following line was missing from:

```
/etc/slurm/slurm.conf
```

Missing configuration:

```
AccountingStorageEnforce=limits,qos
```

Without this line:

- accounting works
- sacct works
- qos appears correctly
- associations appear correctly

BUT:

```
scheduler policies are NOT enforced
```

This is one of the most common beginner Slurm configuration mistakes.

## 20.16 Enable QoS Enforcement

Edit configuration on master node:

```
sudo vim /etc/slurm/slurm.conf
```

Add:

```
AccountingStorageEnforce=limits,qos
```

Recommended placement:

Near existing accounting configuration:

```
AccountingStorageType=accounting_storage/slurmdbd
```

```
AccountingStorageHost=master
```

```
AccountingStoragePort=6819
```

```
JobAcctGatherType=jobacct_gather/linux
```

## 20.17 IMPORTANT Enforcement Concept

This line enables actual scheduler-side enforcement for:

- qos limits
- account limits
- user policies
- scheduler restrictions

Without:

`AccountingStorageEnforce`

QoS becomes informational only.

## 20.18 Distribute Updated Configuration

Copy updated configuration from master node:

```
scp /etc/slurm/slurm.conf login:/tmp/  
scp /etc/slurm/slurm.conf compute01:/tmp/  
scp /etc/slurm/slurm.conf compute02:/tmp/  
scp /etc/slurm/slurm.conf compute03:/tmp/
```

Move configuration into place on login node:

```
sudo mv /tmp/slurm.conf /etc/slurm/
```

Move configuration into place on compute nodes:

```
ssh compute01 sudo -S mv /tmp/slurm.conf /etc/slurm/  
ssh compute02 sudo -S mv /tmp/slurm.conf /etc/slurm/  
ssh compute03 sudo -S mv /tmp/slurm.conf /etc/slurm/
```

## 20.19 Restart Slurm Services

Restart controller on master node:

```
sudo systemctl restart slurmctld
```

Restart compute daemons:

```
ssh compute01 sudo -S systemctl restart slurmd
ssh compute02 sudo -S systemctl restart slurmd
ssh compute03 sudo -S systemctl restart slurmd
```

Purpose:

Reload updated scheduler policy configuration.

## 20.20 Verify Cluster Health

Run:

```
sinfo
```

Expected:

```
compute[01-03] idle
```

This validates:

- scheduler stability
- successful configuration reload
- healthy compute nodes

## 20.21 Verify QoS Enforcement

### 1. CPU Limit Validation

Job requesting:

6 CPUs

using:

```
#SBATCH -N 3
#SBATCH --ntasks-per-node=2
#SBATCH --ntasks=6
```

was submitted.

Observed scheduler behavior:

```
PENDING
Reason=QOSMaxCpuPerUserLimit
```

This validates:

- active QoS enforcement
- CPU limit enforcement
- scheduler-side policy restriction

## 2. IMPORTANT CPU QoS Concept

```
MaxTRESPerUser=cpu=4
```

means:

maximum ACTIVE running CPUs simultaneously

NOT:

- maximum job submissions
- maximum queued jobs
- maximum CPUs requested in scripts

This distinction is extremely important.



## 20.22 Verify Valid CPU Allocation

Job requesting:

4 CPUs

using:

```
#SBATCH -N 2
#SBATCH --ntasks-per-node=2
```

executed successfully.

Observed output:

```
SLURM_NTASKS=4
SLURM_JOB_NUM_NODES=2
SLURM_CPUS_ON_NODE=2
```

This validates:

- multinode allocation
- scheduler variable propagation
- QoS-aware MPI execution
- proper CPU accounting

## 20.23 IMPORTANT MPI + QoS Integration

MPI jobs remained fully functional while QoS policies were enforced.

This validates successful integration between:

- Slurm scheduler
- QoS policies
- multinode scheduling
- MPI runtime
- resource accounting

## 20.24 Verify Walltime Enforcement

Job script modified:

```
#SBATCH --time=02:00:00
```

Observed scheduler behavior:

```
PENDING
```

```
Reason=QOSMaxWallDurationPerJobLimit
```

This validates:

- runtime limit enforcement
- scheduler-side walltime policy control

## 20.25 IMPORTANT Pending Queue Concept

QoS-restricted jobs are usually:

queued and pending

NOT:

immediately rejected

This is standard production HPC scheduler behavior.  
Scheduler explains pending reason using:

```
squeue --me
```

or:

```
scontrol show job <jobid>
```

This greatly improves scheduler transparency.

## 20.26 Inspect QoS Configuration

View configured QoS settings:

```
sacctmgr show qos normal
```

Observed configuration:

```
MaxWall=01:00:00
```

```
MaxTRESPU=cpu=4
```

```
MaxJobsPU=2
```

This validates:

- correct QoS policy configuration
- scheduler policy persistence
- accounting database integration

## 20.27 IMPORTANT Scheduler Layers

The cluster now supports multiple scheduler layers simultaneously.

| Layer               | Purpose                          |
|---------------------|----------------------------------|
| Resource scheduling | CPU and memory allocation        |
| Affinity scheduling | CPU placement and pinning        |
| Accounting          | Persistent job/resource tracking |
| Policy enforcement  | User and QoS restrictions        |

This architecture is significantly closer to real production HPC clusters.

## 20.28 Current Cluster Capability

The cluster now supports:

| Feature                        | Status  |
|--------------------------------|---------|
| Multinode scheduling           | Working |
| MPI execution                  | Working |
| LDAP authentication            | Working |
| Shared filesystem              | Working |
| Spack software stack           | Working |
| Resource-aware scheduling      | Working |
| CPU affinity                   | Working |
| Persistent accounting          | Working |
| QoS enforcement                | Working |
| User limits                    | Working |
| CPU policy enforcement         | Working |
| Walltime policy enforcement    | Working |
| Multi-user scheduling policies | Working |

The cluster has now evolved into a significantly more realistic multi-user HPC scheduling environment.

## 21 Validation and Verification of Advanced Slurm Configuration and QoS

This section validates all advanced Slurm scheduler features configured during the setup process.

The purpose of validation is to ensure:

- scheduler stability
- correct resource tracking
- proper QoS enforcement
- multinode execution
- MPI integration
- accounting consistency
- policy-aware scheduling behavior

All validation steps were tested practically on the configured cluster.

## 21.1 Cluster Validation State

At the time of validation, the cluster already supported:

| Feature                   | Status  |
|---------------------------|---------|
| Slurm scheduler           | Working |
| Multinode scheduling      | Working |
| MPI execution             | Working |
| LDAP authentication       | Working |
| Shared filesystem         | Working |
| Spack software stack      | Working |
| Resource-aware scheduling | Working |
| CPU affinity              | Working |
| Accounting database       | Working |
| QoS enforcement           | Working |

This validation phase ensures all components function together correctly.

## 21.2 Verify Cluster Health

Run on login node:

```
sinfo
```

Expected:

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*      up    infinite     3   idle compute[01-03]
```

Purpose:

Verifies:

- scheduler communication
- node availability
- slurmd daemon health
- cluster operational state

IMPORTANT:

Nodes should NOT show:

- drain

- down
- fail
- unknown

during normal operation.

### 21.3 Verify Node Resource Recognition

Run:

```
scontrol show node compute01
```

Important fields:

CPUTot=2

RealMemory=1200

CfgTRES=cpu=2,mem=1200M

Purpose:

Validates:

- CPU detection
- memory-aware scheduling
- TRES resource tracking
- scheduler resource recognition

This confirms Slurm now understands:

- schedulable CPUs
- schedulable memory
- consumable resources

instead of only node allocation.

## 21.4 Verify Accounting Infrastructure

Run:

```
sudo systemctl status mariadb
```

Expected:

**active (running)**

Purpose:

Validates accounting database backend.

Verify accounting daemon:

```
sudo systemctl status slurmdbd
```

Expected:

**active (running)**

Purpose:

Validates:

- Slurm database daemon
- scheduler/accounting communication
- persistent accounting infrastructure

## 21.5 Verify Cluster Accounting Registration

Run:

```
sacctmgr list cluster
```

Expected:

**hpccluster**

Purpose:

Verifies:

- cluster registration
- accounting database connectivity
- slurmdbd integration

## 21.6 Verify Accounts and User Associations

Run:

```
sacctmgr show assoc
```

Expected:

| Cluster    | Account | User  | Partition | QoS    |
|------------|---------|-------|-----------|--------|
| hpccluster | general | rishi |           | normal |

Purpose:

Validates:

- Slurm account creation
- user associations
- QoS assignments
- scheduler policy hierarchy

## 21.7 Verify QoS Configuration

Run:

```
sacctmgr show qos normal
```

Expected:

```
MaxWall=01:00:00
```

```
MaxTRESPU=cpu=4
```

```
MaxJobsPU=2
```

Purpose:

Validates:

- QoS creation
- CPU limits
- runtime limits
- job count limits



## 21.8 IMPORTANT Validation Issue Faced

Initially:

- QoS existed
- associations existed
- accounting worked
- jobs tracked correctly

BUT:

`scheduler policies were NOT enforced`

Jobs exceeding configured CPU limits still executed successfully.

Example:

`MaxTRESPU=cpu=4`

BUT:

`6 CPU jobs still executed`

This revealed a missing scheduler enforcement configuration.

## 21.9 Root Cause Analysis

The following configuration was missing:

`AccountingStorageEnforce=limits,qos`

inside:

`/etc/slurm/slurm.conf`

Without this line:

- accounting works
- sacct works
- qos visible
- associations visible

BUT:

`scheduler-side policy enforcement remains disabled`

This is one of the most common Slurm QoS configuration mistakes.

## 21.10 Fix Applied

Added to:

```
/etc/slurm/slurm.conf
```

```
AccountingStorageEnforce=limits,qos
```

Then:

- distributed configuration to all nodes
- restarted slurmd
- restarted slurmd daemons

This enabled actual policy enforcement.

## 21.11 MPI Validation Program

The following MPI-based parallel array summation program was used for validation.

Program characteristics:

- MPI parallel execution
- multinode communication
- distributed workload
- scheduler-aware execution
- CPU resource consumption

Compilation:

```
mpicc arrSum.c -o a.out
```

Purpose:

Generate realistic MPI workloads for scheduler validation.

## 21.12 Validation Job Script

Validation batch script:

```
#!/bin/bash
#SBATCH -N 2
#SBATCH --ntasks-per-node=2
#SBATCH --job-name=test
#SBATCH --output=%J.out
#SBATCH --error=%J.err
#SBATCH --time=00:10:00

source /apps/spack/share/spack/setup-env.sh
spack load openmpi

echo "SLURM_NTASKS=$SLURM_NTASKS"
echo "SLURM_JOB_NUM_NODES=$SLURM_JOB_NUM_NODES"
echo "SLURM_CPUS_ON_NODE=$SLURM_CPUS_ON_NODE"

mpirun -np $SLURM_NTASKS ./a.out
```

Purpose:

Validates:

- multinode scheduling
- MPI integration
- scheduler environment variables
- CPU allocation
- QoS-aware execution

## 21.13 Verify Successful MPI Execution

Submit job:

```
sbatch submit.sh
```

Monitor queue:

```
squeue --me
```

Expected:

RUNNING

Purpose:

Verifies:

- scheduler functionality
- multinode allocation
- successful MPI execution

## 21.14 Verify Scheduler Environment Variables

Observed output:

```
SLURM_NTASKS=4
SLURM_JOB_NUM_NODES=2
SLURM_CPUS_ON_NODE=2
```

This validates:

- correct CPU allocation
- correct task allocation
- proper multinode scheduling
- scheduler environment propagation

## 21.15 Verify CPU QoS Enforcement

Modified job script:

```
#SBATCH -N 3
#SBATCH --ntasks-per-node=2
#SBATCH --ntasks=6
```

Requested resources:

6 CPUs

Configured QoS limit:

MaxTRESPU=cpu=4

Observed scheduler behavior:

PENDING

Reason=QOSMaxCpuPerUserLimit

Purpose:

Validates:

- CPU policy enforcement
- active QoS restriction
- scheduler-side resource control

## 21.16 IMPORTANT CPU QoS Concept

MaxTRESPU=cpu=4

means:

maximum ACTIVE running CPUs simultaneously

NOT:

- maximum submitted jobs
- maximum queued jobs
- maximum CPUs requested in scripts

This distinction is extremely important.

## 21.17 Verify Walltime Enforcement

Modified batch script:

```
#SBATCH --time=02:00:00
```

Configured QoS limit:

MaxWall=01:00:00

Observed scheduler behavior:

PENDING

Reason=QOSMaxWallDurationPerJobLimit

Purpose:

Validates:

- runtime limit enforcement
- walltime policy enforcement
- scheduler-side runtime restriction

## 21.18 Verify Job Count Enforcement

Configured:

MaxJobsPU=2

Submitted multiple jobs:

```
sbatch submit.sh
sbatch submit.sh
sbatch submit.sh
```

Observed behavior:

- first jobs execute
- additional jobs remain pending

Purpose:

Validates:

- concurrent job restriction
- scheduler fairness policy
- user job limit enforcement

## 21.19 Verify Accounting Persistence

Run:

```
sacct
```

Observed output:

| JobID    | JobName | Partition | AllocCPUS | State     |
|----------|---------|-----------|-----------|-----------|
| 22       | test    | debug     | 4         | COMPLETED |
| 22.batch | batch   |           | 2         | COMPLETED |
| 22.0     | prted   |           | 2         | COMPLETED |

Purpose:

Validates:

- persistent accounting
- MPI runtime tracking
- historical scheduler data
- job execution tracking

## 21.20 IMPORTANT MPI Accounting Observation

Accounting tracks internal MPI execution hierarchy:

| Entry    | Meaning              |
|----------|----------------------|
| 22       | main scheduler job   |
| 22.batch | batch execution step |
| 22.0     | MPI runtime launcher |

This provides deeper scheduler visibility similar to production HPC environments.

## 21.21 Final Validation State

After validation, the cluster successfully supported:

| Feature                       | Status  |
|-------------------------------|---------|
| Multinode scheduling          | Working |
| MPI execution                 | Working |
| LDAP authentication           | Working |
| Shared filesystem             | Working |
| Spack software stack          | Working |
| Resource-aware scheduling     | Working |
| CPU affinity                  | Working |
| Accounting database           | Working |
| Persistent accounting         | Working |
| QoS enforcement               | Working |
| CPU policy enforcement        | Working |
| Runtime policy enforcement    | Working |
| User job limits               | Working |
| Scheduler-side policy control | Working |

## 21.22 Validation Conclusion

The cluster successfully evolved from:

`basic educational scheduler`

into:

`policy-aware production-like HPC infrastructure`

while maintaining:

- scheduler stability
- multinode MPI execution
- resource-aware scheduling
- persistent accounting
- user policy enforcement
- simplified administration

This now closely resembles the operational structure used in real HPC centers and supercomputing environments.



## 22 Fairshare Scheduling and Multifactor Priority Scheduling

This phase upgrades the Slurm scheduler from:

`policy-aware scheduling`

into:

`fairness-aware scheduling`

At this stage, the cluster already supported:

- multinode scheduling
- MPI execution
- resource-aware scheduling
- QoS enforcement
- persistent accounting
- CPU-aware scheduling
- memory-aware scheduling
- user policy enforcement

The goal of this phase was to introduce:

- fair resource sharing
- historical usage balancing
- multifactor scheduling
- scheduler priorities
- usage-aware scheduling behavior

similar to real production HPC environments.

## 22.1 IMPORTANT Fairshare Concept

Before enabling fairshare:

Scheduler decisions were mostly based on:

- available resources
- QoS limits
- job submission order

Meaning:

`first user to submit jobs usually gains resources first`

This can create unfair long-term resource usage.

Fairshare introduces:

`historical usage-aware scheduling`

Meaning:

Users consuming large amounts of resources gradually lose scheduler priority while lighter users gain scheduling preference.

This creates:

- balanced cluster usage
- long-term fairness
- anti-resource-hoarding behavior
- improved scheduler fairness

Production supercomputers heavily rely on fairshare scheduling.

## 22.2 IMPORTANT Difference Between QoS and Fairshare

QoS controls:

`hard scheduling limits`

Examples:

- maximum CPUs

- maximum jobs
- maximum runtime

Fairshare controls:

dynamic scheduling priority balancing

Meaning:

Jobs are NOT strictly blocked.

Instead:

scheduler priority changes over time

depending on:

- historical resource usage
- waiting time
- scheduler fairness policies

This distinction is extremely important.

## 22.3 Existing Stable Scheduler Infrastructure

Before enabling fairshare, the cluster already supported:

| Feature                      | Status  |
|------------------------------|---------|
| Multinode scheduling         | Working |
| MPI execution                | Working |
| QoS enforcement              | Working |
| Resource-aware scheduling    | Working |
| CPU affinity                 | Working |
| Persistent accounting        | Working |
| Slurm database accounting    | Working |
| Scheduler policy enforcement | Working |

This provided the required foundation for multifactor scheduling.

## 22.4 IMPORTANT Scheduler Concept

Fairshare requires:

`multifactor priority scheduling`

because scheduler decisions must now consider multiple factors simultaneously.

Without multifactor scheduling:

`fairshare` cannot function correctly

## 22.5 Enable Multifactor Priority Scheduler

Edit Slurm configuration on master node:

```
sudo vim /etc/slurm/slurm.conf
```

Add:

```
PriorityType=priority/multifactor
```

```
PriorityWeightFairshare=10000
```

```
PriorityWeightAge=1000
```

```
PriorityWeightJobSize=100
```

```
PriorityWeightPartition=1000
```

Recommended placement:

Near existing scheduler configuration:

```
SchedulerType=sched/backfill
```

## 22.6 IMPORTANT PriorityType Concept

```
PriorityType=priority/multifactor
```

enables:

`advanced scheduler priority calculations`

The scheduler now computes priorities using:

- `fairshare`

- waiting age
- job size
- partition priority
- scheduler weights

This is the standard production HPC scheduling model.

## 22.7 Understanding PriorityWeightFairshare

`PriorityWeightFairshare=10000`

This is the MOST important fairshare configuration line.

Purpose:

Controls how strongly historical resource usage influences scheduler priority.

Higher value:

**stronger fairshare effect**

Meaning:

Users with lower historical usage gain significantly better priority.

## 22.8 Understanding PriorityWeightAge

`PriorityWeightAge=1000`

Purpose:

Older waiting jobs gradually gain scheduler priority.

Prevents:

**scheduler starvation**

Meaning:

Jobs waiting for long periods slowly move upward in priority.

## 22.9 Understanding PriorityWeightJobSize

PriorityWeightJobSize=100

Purpose:

Influence scheduling based on requested job size.

Depending on cluster policy:

- larger jobs may receive preference
- smaller jobs may backfill easier

This helps improve cluster utilization.

## 22.10 Understanding PriorityWeightPartition

PriorityWeightPartition=1000

Purpose:

Allow different partitions to receive different scheduler priorities.

Useful for future partitions such as:

- debug
- teaching
- long-running
- premium
- GPU partitions

## 22.11 Configure Fairshare Decay Settings

Still inside:

/etc/slurm/slurm.conf

Add:

PriorityDecayHalfLife=7-0

PriorityUsageResetPeriod=NONE

PriorityFavorSmall=NO

FairShareDampeningFactor=1

## 22.12 Understanding PriorityDecayHalfLife

PriorityDecayHalfLife=7-0

Meaning:

Historical resource usage gradually decays over:

7 days

Purpose:

Allows users to recover scheduler priority over time.

Without decay:

historical usage penalties become permanent

which is undesirable.

Production clusters commonly use:

- days
- weeks
- months

depending on scheduling policy.

## 22.13 Understanding PriorityUsageResetPeriod

PriorityUsageResetPeriod=NONE

Meaning:

Usage history never fully resets.

Instead:

usage gradually decays naturally

This creates smoother long-term fairness behavior.

## 22.14 Understanding PriorityFavorSmall

PriorityFavorSmall=NO

Purpose:

Prevent scheduler from aggressively favoring very small jobs.

This keeps scheduling behavior more balanced.

## 22.15 Understanding FairShareDampeningFactor

FairShareDampeningFactor=1

Purpose:

Controls strength of fairshare effects.

Lower values:

- weaker balancing

Higher values:

- stronger balancing

Value:

1

provides moderate balanced behavior.

## 22.16 Distribute Updated Configuration

Copy updated configuration from master node:

```
scp /etc/slurm/slurm.conf login:/tmp/  
scp /etc/slurm/slurm.conf compute01:/tmp/  
scp /etc/slurm/slurm.conf compute02:/tmp/  
scp /etc/slurm/slurm.conf compute03:/tmp/
```

Purpose:

Distribute updated scheduler configuration to all nodes.

## 22.17 Install Configuration

Move configuration into place on login node:

```
sudo mv /tmp/slurm.conf /etc/slurm/
```

Move configuration into place on compute nodes:

```
ssh compute01 sudo -S mv /tmp/slurm.conf /etc/slurm/  
ssh compute02 sudo -S mv /tmp/slurm.conf /etc/slurm/  
ssh compute03 sudo -S mv /tmp/slurm.conf /etc/slurm/
```

Purpose:

Replace old scheduler configuration with updated multifactor scheduling configuration.



## 22.18 Restart Slurm Services

Restart scheduler controller on master node:

```
sudo systemctl restart slurmctld
```

Observed temporarily:

```
compute[01-03] unk
```

This is normal during scheduler restart.

Restart compute daemons:

```
ssh compute01 sudo -S systemctl restart slurmd
ssh compute02 sudo -S systemctl restart slurmd
ssh compute03 sudo -S systemctl restart slurmd
```

After restart:

```
sinfo
```

Observed:

```
compute[01-03] idle
```

This validated:

- successful scheduler reload
- healthy compute nodes
- stable scheduler state

## 22.19 IMPORTANT Issue Initially Observed

Immediately after configuration:

```
sprio
```

returned:

You are not running a supported priority plugin

This revealed:

PriorityType=priority/multifactor

was not yet active.

Cause:

Scheduler restart had not yet fully applied the new configuration.

After:

- configuration distribution
- slurmd restart
- slurmctld restart

the issue disappeared successfully.

## 22.20 IMPORTANT sprio Concept

sprio

is one of the MOST important Slurm administrator commands.

Purpose:

Displays:

- scheduler priority calculations
- fairshare contribution
- job age contribution
- job size contribution
- partition priority contribution

This command explains:

WHY jobs receive current scheduling priority

## 22.21 IMPORTANT Observation About sprio

Initially:

`sprio`

showed no jobs.

This was NOT a failure.

Reason:

`sprio` only displays pending jobs

NOT:

- completed jobs
- running jobs

Since jobs executed quickly, there were no pending jobs to analyze.

This behavior is completely normal.

## 22.22 Verify Fairshare Accounting

Run:

`sshare`

Observed output:

`RawUsage = 387`

Purpose:

Validates:

- historical usage tracking
- accounting integration
- fairshare accounting functionality

This proved Slurm was successfully recording:

- CPU usage
- historical resource consumption
- scheduling history

## 22.23 Understanding sshare Output

`sshare`

displays:

- scheduler shares
- historical usage
- fairshare values
- normalized shares

This is one of the most important scheduler accounting commands.

## 22.24 Understanding RawUsage

Example:

`RawUsage = 387`

Meaning:

User has consumed historical scheduler resources.

This includes:

- CPU usage
- scheduler allocations
- accumulated job execution time

Higher value means:

`heavier historical cluster usage`

## 22.25 Understanding FairShare Value

Example:

`FairShare = 1.000000`

Meaning:

User currently has full scheduling entitlement.

Initially this remained:

1.000000

because only one active user existed.  
Fairshare becomes meaningful only when:

- multiple users compete
- jobs remain pending
- scheduler must prioritize workloads

This is extremely important.

## 22.26 IMPORTANT Fairshare Concept

Fairshare is:

`comparative scheduling behavior`

NOT:

`absolute scheduling behavior`

Meaning:

Fairshare only matters when scheduler must choose between competing jobs.

## 22.27 Create Additional User Competition

Additional users added to Slurm accounting:

```
sudo sacctmgr add user hpcadmin account=general
sudo sacctmgr add user student01 account=general
sudo sacctmgr add user student02 account=general
```

Verify associations:

```
sacctmgr show assoc
```

Purpose:

Create realistic multi-user scheduler competition.

## 22.28 Generate Scheduler Contention

Multiple users submitted jobs simultaneously.

Observed queue state:

squeue

Observed:

```
44 debug test student01 PD (Resources)
45 debug test student01 PD (Priority)
46 debug test rishi      PD (Priority)
```

This was the FIRST real proof that:

multifactor scheduler priorities were functioning

## 22.29 IMPORTANT Understanding Pending Reasons

(Resources)

Meaning:

Cluster currently lacks sufficient free CPUs or nodes.

Scheduler would run the job immediately if resources existed.

(Priority)

Meaning:

Resources may become available, BUT another job currently has higher scheduler priority.

This is the MOST important fairshare scheduling observation.

## 22.30 Understand sprio Output

Observed example:

| JOBID | PARTITION | PRIORITY | SITE | AGE | FAIRSHARE | JOBSIZE | PARTITION |
|-------|-----------|----------|------|-----|-----------|---------|-----------|
| 44    | debug     | 9066     | 0    | 0   | 8000      | 67      | 1000      |
| 46    | debug     | 3066     | 0    | 0   | 2000      | 67      | 1000      |

This output demonstrates active multifactor scheduling.

## 22.31 Understanding PRIORITY Column

Example:

9066

Meaning:

Final scheduler priority score.

Higher value means:

`job executes earlier`

This is the combined result of:

- fairshare
- age
- partition priority
- job size

## 22.32 Understanding FAIRSHARE Column

Example:

8000

Meaning:

Fairshare contribution to total scheduler priority.

Higher value:

- lighter historical usage
- scheduler preference
- better fairness entitlement

Lower value:

- heavier historical resource usage
- reduced scheduler preference

This is the core of fairshare scheduling.

### 22.33 IMPORTANT Fairshare Validation

Observed priorities:

| Job | Fairshare Contribution |
|-----|------------------------|
| 44  | 8000                   |
| 46  | 2000                   |

This proved:

historical resource usage was actively influencing scheduler priority

This is REAL production-style scheduler behavior.

### 22.34 Understanding AGE Column

Initially:

0

because jobs were still new.

Over time:

- waiting jobs accumulate age score
- scheduler gradually boosts their priority

Purpose:

Prevent:

scheduler starvation

### 22.35 Understanding JOBSIZE Column

Example:

67

Meaning:

Scheduler contribution based on requested resources.

Can depend on:

- CPUs



- nodes
- tasks
- requested resources

Different clusters tune this differently.

## 22.36 Understanding PARTITION Column

Example:

1000

This came from:

`PriorityWeightPartition=1000`

Currently all jobs used the same partition.

In future:

Different partitions can receive:

- different priorities
- different scheduling behavior
- different policies

## 22.37 IMPORTANT Final Scheduler State

The scheduler now simultaneously considers:

| Scheduling Factor | Purpose                    |
|-------------------|----------------------------|
| resources         | free CPUs and nodes        |
| qos               | policy restrictions        |
| fairshare         | historical usage balancing |
| age               | waiting time fairness      |
| partition         | queue importance           |
| job size          | workload balancing         |

This is significantly closer to real production HPC scheduling behavior.

## 22.38 Final Fairshare Validation State

After validation, the cluster successfully supported:

| Feature                       | Status  |
|-------------------------------|---------|
| Multifactor scheduling        | Working |
| Fairshare accounting          | Working |
| Historical usage tracking     | Working |
| Dynamic scheduler priorities  | Working |
| Queue priority balancing      | Working |
| QoS integration               | Working |
| Resource-aware scheduling     | Working |
| Pending priority calculations | Working |

## 22.39 Final Conclusion

The scheduler successfully evolved from:

`simple resource-based scheduling`

into:

`fairness-aware multifactor HPC scheduling`

while maintaining:

- cluster stability
- MPI execution
- multinode scheduling
- scheduler accounting
- QoS enforcement
- realistic HPC scheduling behavior

This now resembles the scheduling architecture used in real HPC centers and production supercomputing environments.